

Revisit the Boyar-Peralta Algorithm to Solve the Shortest Linear Program Problem

Yao Sun, RunHe Yang and Ting Li

State Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering,
Chinese Academy of Sciences, Beijing, China.

School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China.

litong@iie.ac.cn

Abstract. The efficiency of circuit implementations for cryptographic algorithms is crucial for their practical deployment. The implementation cost of the linear layer can be evaluated by the number of XOR operations, typically measured in generalized XOR (g-XOR) and serial XOR (s-XOR). Finding the minimal sequence of g-XOR operations constitutes the Shortest Linear Program (SLP) problem. Existing approaches generally address this problem through a two-stage framework: generating an initial sequence followed by local optimization, both stages essentially solving the SLP problem. The Boyar-Peralta (BP) algorithm serves as a foundational heuristic for addressing the SLP problem and has been extensively adopted in subsequent research; however, its computational inefficiency presents a significant limitation for practical applications, especially in large-scale problems. This paper proposes a novel implementation framework for the BP algorithm based on linear combinations, significantly enhancing its computational efficiency. We further introduce a new strategy for selecting subsequences during local optimization, replacing the random selection strategy employed in previous works. By analyzing the structural properties of the Advanced Encryption Standard (AES) linear layer matrix and applying our proposed methods, we achieve the first implementation requiring only 89 g-XOR operations, improving upon the previous best result of 91 g-XOR operations reported in prior literature.

Keywords: Linear Layer · Implementation · XOR Counts · AES · the Boyar-Peralta Algorithm.

1 Introduction

The design of cryptographic components is of importance in cryptographic algorithm design, with lightweight cryptographic component design being a key focus in this field. On the one hand, the growing deployment of resource-constrained devices, such as IoT nodes and RFID tags, has heightened the demand for lightweight cryptographic technologies. This trend has motivated the design of lightweight ciphers, which in turn requires the development of lightweight components [LW16, JPST17, LW17, BLS⁺17, DL18, LSL⁺19, JBKK25], including substitution boxes (S-boxes) and maximum distance separable (MDS) matrices. On the other hand, improving the implementation efficiency of existing cryptographic components remains a significant research challenge. A prominent example is the Advanced Encryption Standard (AES) [DR02], the most widely deployed cipher, for which reducing implementation costs has attracted significant attention [KLSW17, XZL⁺20, LXZZ21, YWS⁺24, JBKK25]. These efforts highlight the importance of optimizing the implementation of cryptographic primitives.

The linear layer, serving as a core component of symmetric cryptography, is typically represented as a binary matrix. The implementation cost of the linear layer can be quantified by the number of XOR operations required. Researchers have introduced two key metrics for optimizing linear layers: generalized XOR (g-XOR) and serial XOR (s-XOR). Consequently, many works aim to minimize the g-XOR or s-XOR count of binary matrices over the Galois Field $\text{GF}(2)$. In this paper, we focus on g-XOR implementations. This is equivalent to the Shortest Linear Programs (SLP) Problem.

The shortest linear program problem Boyar et al. studied the Shortest Linear Program (SLP) problem [BMP08], which is to minimize the number of linear operations necessary to compute a set of linear forms. Here, we provide the mathematical description of the SLP problem.

Problem 1. (The Shortest Linear Program (SLP) Problem) *Let K be a field and K^n be an n -dimensional vector space. Let $B^{(l)} = \{\mathbf{b}_0, \dots, \mathbf{b}_{l-1}\}$ be a set of vectors in K^n , and T be a set of vectors in $\text{Span}_K(B^{(l)})$, where $\text{Span}_K(B^{(l)}) = \{\sum c_i \cdot \mathbf{b}_i \mid c_i \in K, \mathbf{b}_i \in B\}$. The problem is how to find the **minimal set of vectors** $\{\mathbf{b}_l, \mathbf{b}_{l+1}, \dots, \mathbf{b}_{l+s-1}\}$ such that:*

1. *for any $\mathbf{b}_k$ where $l \leq k < l + s$, there always exists $\mathbf{b}_i$ and $\mathbf{b}_j$ s.t. $\mathbf{b}_k = \mathbf{b}_i \oplus \mathbf{b}_j$ where $i, j < k$;*
2. *$T \subseteq \{\mathbf{b}_l, \dots, \mathbf{b}_{l+s-1}\}$.*

The set of vectors $\{\mathbf{b}_l, \mathbf{b}_{l+1}, \dots, \mathbf{b}_{l+s-1}\}$ can be treated as a sequence, which is defined in Section 2, since the order of vectors is a critical factor to consider in the shortest linear program problem.

Minimizing the number of g-Xor operations is equivalent to solving the SLP problem. According to the Definition 2 in [XZL⁺20], each g-XOR represents a single linear operation (XOR) between two vectors. The SLP problem aims to find the minimal sequence of such linear operations, i.e., the fewest number of g-XOR gates. When computing the g-XOR count of a binary matrix, where K is the field $\mathbb{F}_2$, T corresponds to the rows of the matrix, and the vectors in $B^{(l)}$ are unit vectors, the g-XOR counts can be determined by solving the corresponding SLP problem for the matrix.

It is straightforward to find a sequence satisfying two conditions in Problem 1, while finding the shortest such sequence is challenging. Since vectors in T are in $\text{Span}_K(B^{(l)})$, i.e., each vector in T has at least one linear representation with respect to the vectors in $B^{(l)}$. Each such linear representation directly corresponds to a computational sequence.

For example, let $B^{(3)} = \{\mathbf{b}_0, \mathbf{b}_1, \mathbf{b}_2\}$ and $\mathbf{t} = \mathbf{b}_0 \oplus \mathbf{b}_1 \oplus \mathbf{b}_2$. One possible linear representation of $\mathbf{t}$ is $(\mathbf{b}_0 \oplus \mathbf{b}_1) \oplus \mathbf{b}_2$. This representation can be translated into the sequence $\mathcal{B} = \{\mathbf{b}_3, \mathbf{b}_4\}$, where $\mathbf{b}_3 = \mathbf{b}_0 \oplus \mathbf{b}_1$ (a new vector derived from the initial base), and $\mathbf{b}_4 = \mathbf{b}_3 \oplus \mathbf{b}_2$ (which equals $\mathbf{t}$). This sequence $\mathcal{B}$ satisfies the conditions in Problem 1 for the single target $\mathbf{t}$. However, Problem 1 requires a sequence that contains all vectors in T . For multiple target vectors, we can construct a sequence for each target independently and then take the union of all vectors from these sequences. This union will form a set that satisfies both conditions. Nevertheless, this union is generally not minimal. As the vectors in T may have multiple linear representations, the number of sequences increases exponentially in size. Therefore, it is challenging to compute the minimal set of vectors in the shortest linear program problem by exhaustive search. In fact, the shortest linear program problem is NP-hard [BMP08].

Related works The framework to solve the shortest linear program problem has two stages. The first stage involves computing a sequence using the heuristic algorithm, and this sequence may not be optimal. Thus, to obtain a shorter sequence, the second stage involves performing local optimization to improve the results further.

In the first stage, there are two primary technical approaches for generating the initial sequence. One is the exact method, which employs automated modeling frameworks such as Boolean Satisfiability (SAT) [BC20, FSK10, Sto16] for solving. The other is the heuristic approach, primarily represented by the Paar [Paa97] and the Boyar-Peralta (BP) [BP10] algorithms.

Most works of exact methods focus on optimizing the implementation of S-box [Sto16, JPST17, WSL⁺25]. As far as we know, no MILP model for the shortest linear program problem has been published. The reason may be that the MILP model only works for small matrices (e.g., a matrix of size 4×4), as it requires too much memory and computation for larger sizes. However, the linear layer matrices in existing block ciphers are usually of size 16×16 or 32×32 .

In practice, using heuristic algorithms (e.g., the Paar algorithm [Paa97] and the BP algorithm [BP10]) is more realistic than the exact methods. The Paar algorithm [Paa97] is a greedy, locally optimal method for minimizing the number of XOR gates. It iteratively selects the pair of input bits that occurs most frequently across all linear output equations. The Paar algorithm is cancellation-free, i.e., there is no cancellation of variables in the computation. Hence, the Paar algorithm is efficient, but its results are of low quality. The BP algorithm [BP10] introduces the concept of a distance metric when choosing pairs. By considering the distances, the BP algorithm could generate shorter sequences than the Paar algorithm. However, computing the distances is inefficient, so the BP algorithm has not been widely used. In recent years, scholars have improved upon these two classic algorithms, achieving some progress [BFI19, ME19, TP19].

The second stage is dedicated to refining the initial sequence through local optimization. Since the solution from the first stage may be suboptimal, this phase aims to find and eliminate redundant computations. In [TP19, LXZZ21], they introduce several reduction rules to reduce the redundant vectors in sequence for local optimization.

The effectiveness of the above methods is usually demonstrated in the optimization of linear components for critical cryptographic primitives, most notably the AES S-box. The AES S-box, along with its linear transformation layer, has long served as a central benchmark in the quest for minimizing XOR counts in hardware implementations. The previous best results are an implementation of AES MixColumns with only 91 g-XOR operations, achieved through a combination of heuristic methods within a two-stage framework [LXZZ21]. To the best of our knowledge, the previous results of the implementation of AES MixColumns were obtained using general methods without leveraging AES-specific structures.

In summary, current approaches primarily suffer from the following three drawbacks:

1. In the first stage, the BP algorithm can obtain high-quality solutions. However, its relatively low computational efficiency has limited its widespread use.
2. In the second stage, the current strategy for selecting subsequences is entirely random, without considering the characteristics of the sequences. Many subsequences may not have any shorter alternatives; however, the current strategy still tries to optimize them, leading to redundant computations.
3. When applied to specific targets such as the AES MixColumns layer, existing methods do not leverage algorithm-specific structures, thus missing potential optimizations.

Motivations Firstly, we study the BP algorithm and find that its computational bottleneck lies in calculating distances. The value of the reduction in the total distance is a critical criterion for selecting the vector pair to be added to the sequence at each step. The BP algorithm computes distances by exhaustive search, which leads to inefficiency. We investigated the relationship between the distance and the shortest linear representations of the vectors in T . We found that given access to the shortest linear representations, it becomes possible to directly identify the vector pair that yields the most significant

reduction in total distance. This discovery enables a significant acceleration of the BP algorithm. Based on the above, we propose an efficient algorithm that improves the original BP algorithm.

Secondly, we observed that using sequences to represent the target vector generation process is not intrinsic enough. For instance, swapping the positions of specific vectors within a sequence may still yield a sequence that can generate the same target vectors. To address this limitation, we use the graph representation. This graph structure explicitly reveals how many times each vector is utilized in generating the target vectors. Such information is crucial for selecting subsequences during local optimization: vectors that are used less frequently are more likely to be optimizable. This strategy helps reduce redundant computations caused by selecting the subsequence randomly in local optimization.

Finally, we analyzed the structure of the AES MixColumns matrix and observed that specific rows exhibit similar patterns in the positions of 1. Consequently, these similar target vectors can be generated using only a small number of common vectors. This characteristic can be used to improve the efficiency of local optimization.

Contributions This paper focuses on the shortest linear program problem. To solve this problem, we present an efficient algorithm and an MILP-based model for computing the sequence of different problem scales. When the global optimal sequence cannot be directly computed, we propose a new strategy to select subsequences for local optimization. As an application, we analyze the structure of AES MixColumns and apply the aforementioned methods to search for the optimal implementation of the AES MixColumns, and discovered an implementation with a g-XOR count of 89.

To enhance the efficiency of the BP algorithm, we propose the Linear-Combinations-based BP algorithm (LCB-BP algorithm) that utilizes linear combinations. By generating linear combinations of the target vector, this algorithm avoids the inefficient recursive distance calculations in the original BP algorithm, resulting in implementation efficiency. Furthermore, the length of the sequence computed by the LCB-BP algorithm is no larger than that generated by the original BP algorithm. This means any sequence computed by the original BP algorithm can also be found using the LCB-BP algorithm. Additionally, the Paar algorithm is actually a special case of the LCB-BP algorithm.

To avoid redundant computations caused by the random selection strategy, we introduce a new strategy to select subsequences for local optimization. Previously, all existing local optimization approaches selected subsequences of consecutive vectors randomly. In this paper, by presenting sequences as graphs, we could select the vectors having out-degree 1, which are not crucial in sequence generation. The subsequences formed by these vectors can then be replaced with shorter ones.

To compare our approach with existing methods, we evaluated it on several matrices. The results show that our approach yields comparable or better implementations in most cases. Specifically, we analyze the AES MixColumns matrix and improve the methods based on the observations. The improvement leads to an implementation requiring only 89 g-XOR operations as shown in Table 4, which improves upon the previous best record of 91 g-XOR operations achieved by Lin et al. [LXZZ21].

Structure of the Paper This paper is organized as follows. Section 2 introduces necessary notations and preliminaries. Section 3 proposes an exact MILP-based model for the shortest linear program problem, and a new efficient heuristic algorithm based on the BP-algorithm is given. In Section 4, we describe the mathematical problem of sub-graph reconstruction in the local optimization stage and the method to solve this problem; a strategy for selecting the subsequence is presented as well. The applications come in Section 5. We conclude this paper in Section 6.

2 Preliminaries

First, we present some notations and definitions used throughout the paper, and then we introduce the Boyar-Peralta (BP) [BP10] algorithms.

2.1 Notations

Let $\mathbb{F}_2$ be the Galois Field GF(2) and $\mathbb{F}_2^n$ denote the n -dimensional vector space over $\mathbb{F}_2$. We use a lowercase boldface letter (e.g. $\mathbf{b} = (b_0, b_1, \dots, b_n) \in \mathbb{F}_2^n$) to represent a vector. The weight of a vector $\mathbf{b}$, denoted by $w(\mathbf{b})$, is the number of nonzero coordinates (i.e., the number of 1's) in $\mathbf{b}$. The unit vector with a one at the i -th coordinate is denoted by $\mathbf{e}_i$. The vector $\mathbf{b}$ is a linear combination of the vectors $\mathbf{v}_0, \dots, \mathbf{v}_l$ if $\mathbf{b} = c_0 \cdot \mathbf{v}_0 \oplus \dots \oplus c_l \cdot \mathbf{v}_l$ where $c_i \in \mathbb{F}_2$. We say the set of vectors $L = \{\mathbf{v}_i \mid c_i = 1\}$ is an linear representation of $\mathbf{b}$, i.e., $\mathbf{b} = \bigoplus_{\mathbf{v} \in L} \mathbf{v}$ if L is an linear representation of $\mathbf{b}$. The length of the linear representation L , denoted $|L|$, is the number of elements in L .

We use $\mathcal{B} = [\mathbf{b}_0, \mathbf{b}_1, \dots, \mathbf{b}_l]$ to denote a **sequence**, which is an ordered list of vectors. The set of elements in $\mathcal{B}$ is denoted by $\text{set}(\mathcal{B}) = \{\mathbf{b}_0, \mathbf{b}_1, \dots, \mathbf{b}_l\}$. The length of a sequence $\mathcal{B}$ is the number of elements in $\mathcal{B}$, denoted as $|\mathcal{B}|$. For two sequences $\mathcal{A} = [\mathbf{a}_0, \mathbf{a}_1, \dots, \mathbf{a}_h]$ and $\mathcal{B} = [\mathbf{b}_0, \mathbf{b}_1, \dots, \mathbf{b}_l]$, the concatenation of $\mathcal{A}$ and $\mathcal{B}$ is denoted by $\mathcal{A} \mid \mathcal{B} = [\mathbf{a}_0, \mathbf{a}_1, \dots, \mathbf{a}_h, \mathbf{b}_0, \mathbf{b}_1, \dots, \mathbf{b}_l]$.

We refer to a sequence satisfying the two conditions in Problem 1 as a complete vector sum sequence. Its definition is provided as follows.

Definition 1. (Complete vector sum sequence) Let $B^{(l)} = \{\mathbf{b}_0, \dots, \mathbf{b}_{l-1}\}$ be a set of vectors, we call $\mathcal{B}^{(s)} = [\mathbf{b}_l, \dots, \mathbf{b}_{l+s-1}]$ the **vector sum sequence** with respect to $B^{(l)}$ if for each $\mathbf{b}_k \in \mathcal{B}^{(s)}$, there exist $\mathbf{b}_i$ and $\mathbf{b}_j$ s.t. $\mathbf{b}_k = \mathbf{b}_i \oplus \mathbf{b}_j$ where $i, j < k$. We define the **complete vector sum sequence** for T with respect to $B^{(l)}$ to be the sequence $\mathcal{B}^{(s)} = [\mathbf{b}_l, \dots, \mathbf{b}_{l+s-1}]$, if $\mathcal{B}^{(s)}$ is a vector sum sequence with respect to $B^{(l)}$ and $T \subseteq \text{set}(\mathcal{B}^{(s)})$.

When no ambiguity arises, we also denote $\mathcal{B}^{(l)}$ by $\mathcal{B}$ for simplicity. Let $M_{m \times n}$ denote an $m \times n$ matrix over $\mathbb{F}_2$. We represent its rows as a set of target vectors $T = \{\mathbf{t}_0, \mathbf{t}_1, \dots, \mathbf{t}_{m-1}\}$ and called the **output target set**, where each $\mathbf{t}_i$ is an n -dimensional vector corresponding to the i -th row of $M_{m \times n}$. The vector used to generate the target vectors is called the **generator**, and the set of generators is called **input generator set**.

To calculate the g-XOR count of $M_{m \times n}$, we need to find a complete vector sum sequence $\mathcal{B}$ for T with respect to the input generator set $B^{(n)} = \{\mathbf{e}_0, \dots, \mathbf{e}_{n-1}\}$. For every such complete vector sum sequence, there exists at least one corresponding sequence of XOR operations, and the g-XOR count is $|\mathcal{B}|$.

2.2 The Boyar-Peralta Algorithm

The Boyar-Peralta algorithm (BP algorithm) [BP10] is a heuristic greedy algorithm and it builds a complete vector sum sequence $\mathcal{B}$ of base vectors step by step. At each step, $\mathcal{B}$ is extended by concatenating a new vector, potentially reducing the **distance** to the target vectors. Given a set of vectors B and a target vector $\mathbf{t}$, the distance from B to $\mathbf{t}$, denoted by $\delta(B, \mathbf{t})$, is the minimum number of XOR operations required to compute $\mathbf{t}$ starting from vectors in B . Here, we give the definitions of distance.

Definition 2. (Distance) Given a target vector $\mathbf{t}$ and a set of vectors B , the set of all linear representations of $\mathbf{t}$ with respect to B is defined as:

$$\text{LinSet}(B, \mathbf{t}) = \{L \mid \mathbf{t} = \bigoplus_{\mathbf{b} \in L} \mathbf{b}, L \subseteq B\}.$$

The *distance* from B to $\mathbf{t}$ is defined as the minimal length of all linear representations minus one:

$$\delta(B, \mathbf{t}) = \min\{|L| - 1 \mid L \in \text{LinSet}(B, \mathbf{t})\}.$$

Given a set of target vectors $T = \{\mathbf{t}_0, \mathbf{t}_1, \dots, \mathbf{t}_m\}$, its *distance vector* with respect to B is defined as $\text{Dist}(B) = (\delta(B, \mathbf{t}_0), \dots, \delta(B, \mathbf{t}_m))$.

Algorithm 1 The pseudocode of the Boyar-Peralta algorithm

Require: The input generator set $B^{(l)} = \{\mathbf{b}_0, \dots, \mathbf{b}_{l-1}\}$. The output target set $T = \{\mathbf{t}_0, \dots, \mathbf{t}_{m-1}\}$.

Ensure: A complete vector sum sequence $\mathcal{B}$ for T with respect to $B^{(l)}$.

```

1:  $\text{Dist} \leftarrow (w(\mathbf{t}_0) - 1, \dots, w(\mathbf{t}_m) - 1)$ 
2:  $\text{TotalDist} \leftarrow \sum_{d \in \text{Dist}} d$ 
3:  $\mathcal{B} \leftarrow$  an empty sequence
4: while  $\text{TotalDist} \neq 0$  do
5:    $B \leftarrow B^{(l)} \cup \text{set}(\mathcal{B})$ 
6:    $P \leftarrow \{\{\mathbf{b}_i, \mathbf{b}_j\} \mid \mathbf{b}_i, \mathbf{b}_j \in B\}$ 
7:    $D_{\min} \leftarrow \text{TotalDist}$ 
8:    $N_{\max} \leftarrow 0$ 
9:   for  $\{\mathbf{b}_i, \mathbf{b}_j\} \in P$  do
10:     $\mathbf{b}_k = \mathbf{b}_i \oplus \mathbf{b}_j$ 
11:     $\text{Dist} \leftarrow \text{updateDist}(T, B, \mathbf{b}_k, \text{Dist})$ 
12:    if there exists  $d_i = 0$  where  $d_i \in \text{Dist}$  then ▷ “pre-emptive” choice
13:       $\mathcal{B} \leftarrow \mathcal{B} \mid [\mathbf{b}_k]$ 
14:      break
15:    else
16:       $\text{TotalDist} \leftarrow \sum_{d \in \text{Dist}} d$ 
17:       $\text{DistNorm} \leftarrow \sum_{d \in \text{Dist}} d^2$ 
18:      if  $\text{TotalDist} = 0$  then
19:        return  $\mathcal{B} \leftarrow \mathcal{B} \mid [\mathbf{b}_k]$ 
20:      end if
21:      if  $\text{TotalDist} < D_{\min}$  then
22:         $\text{Candidates} \leftarrow \{\mathbf{b}_k\}$ 
23:         $D_{\min} = \text{TotalDist}$ 
24:         $N_{\max} = \text{DistNorm}$ 
25:      end if
26:      if  $\text{TotalDist} = D_{\min}$  and  $\text{DistNorm} > N_{\max}$  then
27:         $\text{Candidates} \leftarrow \{\mathbf{b}_k\}$ 
28:         $N_{\max} = \text{DistNorm}$ 
29:      else if  $\text{TotalDist} = D_{\min}$  and  $\text{DistNorm} = N_{\max}$  then
30:         $\text{Candidates} \leftarrow \text{Candidates} \cup \{\mathbf{b}_k\}$ 
31:      end if
32:    end if
33:  end for
34:  Sort set  $\text{Candidates}$  into array  $CList$ 
35:   $\mathcal{B} \leftarrow \mathcal{B} \mid CList[0]$ 
36: end while

```

The BP algorithm establishes the criterion for selecting vector pairs based on distance, and its main procedure is outlined as follows.

1. Enumerate all pairs of basis vectors $(\mathbf{b}_i, \mathbf{b}_j)$ and obtain a new vector $\mathbf{b}_k = \mathbf{b}_i \oplus \mathbf{b}_j$ where $\mathbf{b}_i, \mathbf{b}_j \in B, i < j$. For each pair, compute the distance sum $\text{TotalDist} = \sum_{\mathbf{t} \in T} \delta(B \cup \{\mathbf{b}_k\}, \mathbf{t})$.

2. Select the vector $\mathbf{b}_k$ that minimizes the distance sum $TotalDist$, and add it to the current sequence $\mathcal{B}$.
3. In case of tie, choose the vector that maximises the Euclidean norm of distance, i.e., select $\mathbf{b}_k$ corresponding to $\max |Dist(B \cup \mathbf{b}_k)|_2$ where $Dist(B \cup \mathbf{b}_k) = (\delta(B \cup \mathbf{b}_k, \mathbf{t}_0), \dots, \delta(B \cup \mathbf{b}_k, \mathbf{t}_{m-1}))$ and $|Dist(B \cup \mathbf{b}_k)|_2 = \sqrt{\sum_{i=0}^{m-1} \delta(B \cup \mathbf{b}_k, \mathbf{t}_i)^2}$ for $\mathbf{t}_i \in T$. (Norm criterion)

According to the original implementation of the BP algorithm [BP18], repeatedly executing the original BP algorithm yields deterministic outcomes. It is because when multiple candidate pairs share the same $totalDist$ value, the maximum norm criterion is applied. Should multiple pairs further exhibit identical norms, the first pair encountered in a fixed order is selected. Algorithm 2 is the pseudocode of the Boyar-Peralta [BP18].

Algorithm 2 The pseudocode of `updateDist()` function

Require: The set of vectors $B^{(l)} = \{\mathbf{b}_0, \dots, \mathbf{b}_{l-1}\}$. The output target set $T = \{\mathbf{t}_0, \dots, \mathbf{t}_{m-1}\}, \mathbf{t}_i \in \mathbb{F}_2^n$. The vector $\mathbf{b}_k$. The distance vector $Dist = [d_0, \dots, d_{m-1}]$ where d_i is the distance from $B^{(l)}$ to $\mathbf{t}_i$.

Ensure: The distance vector $Dist_k$ of T with respect to $B^{(l)} \cup \{\mathbf{b}_k\}$.

```

1: for  $\mathbf{t}_i \in T$  do
2:   if  $\mathbf{t}_i \in B^{(l)}$  then
3:      $d_i \leftarrow 0$ 
4:   else if reachable( $\mathbf{t}_i \oplus \mathbf{b}_k, d_i - 1, 0, l, B^{(l)}$ ) then
5:      $d_i \leftarrow d_i - 1$ 
6:   end if
7: end for
8:  $Dist_k = (d_0, d_1, \dots, d_m)$ 
9: return  $Dist_k$ 

```

In the implementation of the BP algorithm, the most time-consuming part is calculating the sum of distances. On one hand, the algorithm must evaluate all possible pairs of basis vectors (b_i, b_j) in the set B . On the other hand, the BP algorithm implementations rely on recursively calling the `reachable()` function to determine whether elements in B can linearly represent a target vector t_i . The source code of the `reachable()` function is as follows. In the `reachable()` function, there are five parameters: parameter T is the target vector, K is the exact number of basis vectors allowed to compute T , S is the starting index in the basis, $Base$ is the set of basis, i.e., B , and $BaseSize$ is the size of the search window in B . Because the `reachable()` function is called recursively, it is inefficient to compute the distance.

Subsequent research on the BP algorithm has focused primarily on refining the heuristic criteria for selecting vector pairs, with almost no improvements made to its computational efficiency. Consequently, various heuristic methods have been developed to resolve ties in the selection process. For instance, the RNBP algorithm [TP19] employs a randomized norm criterion, which has been shown to achieve better results than the original BP algorithm through multiple executions.

```

1 //return true if T is the sum of K elements among
   Base[S..BaseSize-1]
2 bool reachable(uint64_t T, uint64_t K, uint64_t S, uint64_t
   BaseSize, uint64_t *Base){
3   if (__builtin_popcount(T) <= K){ return true; }
4   if (S > BaseSize - 1){ return false; } //exceeded count
5   if (K == 0){ return false; } //this is probably not reached

```



```

6   if (K == 1){
7       for (int i = S; i < BaseSize; i++){
8           if (T == Base[i]){ return true; }
9               return false;
10      }
11  }
12  //consider those sums containing Base[S]
13  if (reachable(T^Base[S], K-1, S+1, BaseSize, Base)){return
    true;}
14  //consider those sums not containing Base[S]
15  if (reachable(T, K, S + 1, BaseSize, Base)){return true;}
16  return false; //not found
17 }

```

Listing 1: Source code of `reachable()` function

3 Methods to solving the shortest linear program problem

In this section, we give two methods to solve the SLP problem. The subsection 3.1 presents an MILP model for solving this problem. An efficient algorithm called the linear combinations based BP algorithm is proposed in subsection 3.2. A model for generating minimal linear combinations is given in subsection 3.3.

3.1 A model for the shortest linear program problem

We could compute the globally optimal solution for the shortest linear program problem through exhaustive search; however, the search space is prohibitively large, making this method impractical. In this subsection, we propose an exact method for the shortest linear program problem based on the MILP technique.

We propose an MILP model to solve the complete vector sum sequence comprising s elements. To find the globally optimal solution for the shortest linear program problem, we initialize s to be $(\sum_{t \in T} (w(t) - 1)) - 1$, then construct and solve the model given in Algorithm 3. If a solution exists, a complete vector sum sequence of length s is obtained. We then decrement s by one and attempt to find a complete vector sum sequence of length $s - 1$. This process is repeated until the model becomes infeasible. The shortest complete vector sum sequence found during this procedure corresponds to the globally optimal solution of the shortest linear program problem.

However, this model is only suitable for small-scale problems. In our experiments, when s is 8, it may takes thousands of seconds to solve the model.

We can search for the global optimal solution to the SLP problem, i.e., the globally shortest sequence, by progressively reducing the value of the input variable s and checking the feasibility of the model at each step. For instance, we can set an initial $s = 10$ and solve this MILP model. If the model yields a solution, it indicates the existence of a sequence of length 10. Subsequently, we set $s = 9$ and solve again. If the model now has no solution, it demonstrates that $\mathcal{B}^{(10)}$ is the global optimal solution.

Our experimental results indicate that for randomly generated matrices with about 50% density, this method can solve the corresponding SLP problem for 8×8 matrices.

3.2 The linear combinations based BP algorithm

In this subsection, we propose a new framework for the BP algorithm based on linear combinations. In our new framework, by utilizing the linear representations, and we no

Algorithm 3 The MILP Model for generating a complete vector sum sequence

Require: The input generator set $B^{(l)} = \{\mathbf{b}_0, \dots, \mathbf{b}_{l-1}\}, \mathbf{b}_i \in \mathbb{F}_2^n$. The output target set $T = \{\mathbf{t}_0, \dots, \mathbf{t}_{m-1}\}, \mathbf{t}_i \in \mathbb{F}_2^n$. A positive integer s .

Ensure: A complete vector sum sequence $\mathcal{B}^{(s)} = \{\mathbf{b}_l, \dots, \mathbf{b}_{l+s-1}\}$.

- 1: Prepare an empty MILP Model $\mathcal{M}$
- 2: **for** $k = 0$ to $s - 1$ **do**
- 3: $\mathcal{M}.var \leftarrow c[i]$ as binary variables for $0 \leq i < l + k$
- 4: **for** $j = 0$ to $n - 1$ **do**
- 5: $\mathcal{M}.var \leftarrow N$ as a binary variable
- 6: $\mathcal{M}.var \leftarrow \mathbf{b}_{l+k}[j]$ as binary variables $\triangleright \mathbf{b}_{l+k}[j]$ is the j -th bit in $\mathbf{b}_k$
- 7: $\mathcal{M}.con \leftarrow \sum_{i=0}^{l+k-1} c[i] \cdot \mathbf{b}_i[j] = \mathbf{b}_{l+k}[j] + 2 \cdot N$
- 8: **end for** $\mathcal{M}.con \leftarrow \sum_{i=0}^{l+k-1} c[i] = 2$
- 9: **end for**
- 10: **for** $\mathbf{t} \in T$ **do**
- 11: $\mathcal{M}.var \leftarrow c[i]$ as binary variables for $0 \leq i < l + s$
- 12: **for** $j = 0$ to n **do**
- 13: $\mathcal{M}.var \leftarrow N$ as a binary variable
- 14: $\mathcal{M}.con \leftarrow \sum_{i=0}^{l+s-1} c[i] \cdot \mathbf{b}_i[j] = \mathbf{t}[j] + 2 \cdot N,$ $\triangleright \mathbf{t}[j]$ is the j -th bit in $\mathbf{t}$
- 15: **end for**
- 16: $\mathcal{M}.con \leftarrow \sum_{i=0}^{l+s-1} c[i] = 1$
- 17: **end for**
- 18: $\mathcal{M}.optimize()$
- 19: **if** $\mathcal{M}$ has a solution **then**
- 20: $\mathcal{B}^{(s)} \leftarrow [\mathbf{b}_l, \dots, \mathbf{b}_{l+s-1}]$
- 21: **else**
- 22: return NULL
- 23: **end if**
- 24: **return** $\mathcal{B}^{(s)}$

longer need to compute the distance for each $(\mathbf{b}_i, \mathbf{b}_j)$. In fact, distance is determined by the length of the minimal linear representation, which is defined in Definition 3.

Definition 3. (The minimal linear representation) Given a vector $\mathbf{t}$ and a set of vectors B , let $\text{LinSet}(B, \mathbf{t}) = \{L \mid \mathbf{t} = \bigoplus_{\mathbf{b} \in L} \mathbf{b}, L \subseteq B\}$ be the set of all linear representations of $\mathbf{t}$ with respect to B , then the linear representation L is defined as *the minimal linear representation* of $\mathbf{t}$ with respect to B if $|L| = \min\{|L| \mid L \in \text{LinSet}(B, \mathbf{t})\}$.

A vector may have multiple minimal linear representations. The set of all minimal linear representations is then denoted by $\text{MinLinSet}(B, \mathbf{t}) = \{L \mid |L| = \delta(B, \mathbf{t}) + 1, L \in \text{LinSet}(B, \mathbf{t})\}$.

Obviously, the distance can be obtained by computing the minimal linear representation, i.e., $\delta(B, \mathbf{t} + 1) = |L| - 1$, where L is a minimal linear representation of $\mathbf{t}$ with respect to B .

The vector pairs that appear in the minimal linear representation of a target vector $\mathbf{t}$ are crucial, as adding them to the existing vector sum sequence is guaranteed to reduce the distance to $\mathbf{t}$. Let L be a minimal linear representation of $\mathbf{t}$ with respect to B , and let $\mathbf{b}_i, \mathbf{b}_j \in L$. By adding the vector $\mathbf{b}_i \oplus \mathbf{b}_j$ to B , we can construct a new minimal linear representation $L^* = (L \setminus \{\mathbf{b}_i, \mathbf{b}_j\}) \cup \mathbf{b}_i \oplus \mathbf{b}_j$, which satisfies $|L^*| = |L| - 1$. Therefore, vector pairs that appear in minimal linear representations play a critical role. We define the set of all such vector pairs as the support, formally defined as follows:

Definition 4. (Support) Let $\mathbf{t}$ be a target vector and B be a set of vectors. The **support** of $\mathbf{t}$ with respect to B is defined as $\text{support}(B, \mathbf{t}) = \{\{\mathbf{b}_i, \mathbf{b}_j\} \mid \exists L \in \text{MinLinSet}(B, \mathbf{t}) \text{ s.t.}$

$\mathbf{b}_i, \mathbf{b}_j \in L$.

Lemma 1 states that if we select a pair from $\text{support}(B, \mathbf{t})$ and add the sum of the pairs into B , then the distance from B to $\mathbf{t}$ decreases by 1.

Lemma 1. *Let $B^{(l)}$ be a set of vectors and $\mathbf{t}$ be a target vector, the pair $\{\mathbf{b}_i, \mathbf{b}_j\}$ is in $\text{support}(B^{(l)}, \mathbf{t})$ if and only if $\delta(B^{(l+1)}, \mathbf{t}) = \delta(B^{(l)}, \mathbf{t}) - 1$ where $B^{(l+1)} = B^{(l)} \cup \{\mathbf{b}_i \oplus \mathbf{b}_j\}$.*

Proof. To prove sufficiency, suppose that there exists a pair $\{\mathbf{b}_i, \mathbf{b}_j\}$ such that $\delta(B^{(l+1)}, \mathbf{t}) = \delta(B^{(l)}, \mathbf{t}) - 1$ where $B^{(l+1)} = B^{(l)} \cup \{\mathbf{b}_i \oplus \mathbf{b}_j\}$ and $\mathbf{b}_i, \mathbf{b}_j \in B^{(l)}$. First, we prove that each minimal linear representation of $\mathbf{t}$ with respect to $B^{(l)} \cup \{\mathbf{b}_i \oplus \mathbf{b}_j\}$ contains the vector $\mathbf{b}_i \oplus \mathbf{b}_j$. For each $L \in \text{MinLinSet}(B^{(l+1)}, \mathbf{t})$, we have $(\mathbf{b}_i \oplus \mathbf{b}_j) \in L$; otherwise, there exist $L' \in \text{MinLinSet}(B^{(l+1)}, \mathbf{t})$ such that $L' \in \text{LinSet}(B^{(l)})$ and $|L'| = \delta(B^{(l+1)}, \mathbf{t}) + 1 = \delta(B^{(l)}, \mathbf{t})$, which leads to a contradiction with the definition of the distance. Thus, we have $\mathbf{b}_i \oplus \mathbf{b}_j \in L$ must hold for every $L \in \text{MinLinSet}(B^{(l+1)}, \mathbf{t})$. Then, we construct a minimal linear representation $L^* \in \text{MinLinSet}(B^{(l)}, \mathbf{t})$ such that $\mathbf{b}_i, \mathbf{b}_j \in L^*$ as follows: $L^* = (L \setminus \{\mathbf{b}_i \oplus \mathbf{b}_j\}) \cup \{\mathbf{b}_i, \mathbf{b}_j\}$. Obviously, L^* is a linear representation of $\mathbf{t}$. We have $L^* \in \text{LinSet}(B^{(l)}, \mathbf{t})$ and $|L^*| = |L| + 1 = \delta(B^{(l+1)}, \mathbf{t}) + 2 = \delta(B^{(l)}, \mathbf{t}) + 1$. Thus, we have $L^* \in \text{MinLinSet}(B^{(l)}, \mathbf{t})$ and $\{\mathbf{b}_i, \mathbf{b}_j\} \in \text{support}(B^{(l)}, \mathbf{t})$.

To prove necessity, assume that the pair $\{\mathbf{b}_i, \mathbf{b}_j\}$ is in $\text{support}(B^{(l)}, \mathbf{t})$. Then there exists a L^* such that $\{\mathbf{b}_i, \mathbf{b}_j\} \in L^*$ and $|L^*| = \delta(B^{(l)}, \mathbf{t}) + 1$. Then we can construct a linear representation of $\mathbf{t}$ with respect to $B^{(l+1)} = B^{(l)} \cup \{\mathbf{b}_i \oplus \mathbf{b}_j\}$ as follows: $L = (L^* \setminus \{\mathbf{b}_i, \mathbf{b}_j\}) \cup \{\mathbf{b}_i \oplus \mathbf{b}_j\}$. The length of L is $\delta(B^{(l)}, \mathbf{t})$. Next, we need to prove that $L \in \text{MinLinSet}(B^{(l+1)}, \mathbf{t})$. Assume that there exist a $L' \in \text{MinLinSet}(B^{(l+1)}, \mathbf{t})$ and $|L'| < |L|$, then we consider the following cases:

Case 1: $\mathbf{b}_i \oplus \mathbf{b}_j \notin L'$. In this case, we have $L' \in \text{LinSet}(B^{(l)}, \mathbf{t})$. Since $|L'| < |L|$ and $|L| = \delta(B^{(l)}, \mathbf{t})$, we have $|L'| < \delta(B^{(l)}, \mathbf{t})$, which leads to a contradiction with the definition of the distance.

Case 2: $\mathbf{b}_i \oplus \mathbf{b}_j \in L'$. In this case, we can construct $\bar{L} = (L' \setminus \{\mathbf{b}_i \oplus \mathbf{b}_j\}) \cup \{\mathbf{b}_i, \mathbf{b}_j\}$ and $\bar{L} \in \text{LinSet}(B^{(l)}, \mathbf{t})$. Then we have $|\bar{L}| = |L'| + 1 < \delta(B^{(l)}, \mathbf{t}) + 1$.

In both cases, there are contradictions with the definition of the distance, and we have $L \in \text{MinLinSet}(B^{(l+1)}, \mathbf{t})$. Thus, $B^{(l+1)} = |L| - 1 = \delta(B^{(l)}, \mathbf{t}) - 1$. $\square$

Lemma 1 considers only a single target vector. The result can be directly generalized to multiple target vectors, as presented in Corollary 1.

Corollary 1. *Let $B^{(l)}$ be a set of vectors and T be a target set. For any subset $T' \subseteq T$ and any pair of vectors $\mathbf{b}_i, \mathbf{b}_j \in B^{(l)}$, the following statements are equivalent:*

1. *The vector pair $\{\mathbf{b}_i, \mathbf{b}_j\}$ belongs to the intersection of the support over T' , i.e., $\{\mathbf{b}_i, \mathbf{b}_j\} \in \bigcap_{\mathbf{t} \in T'} \text{support}(B^{(l)}, \mathbf{t})$.*
2. *For every target vector $\mathbf{t} \in T'$, the distance decreases by one when extending the basis, i.e., $\delta(B^{(l+1)}, \mathbf{t}) = \delta(B^{(l)}, \mathbf{t}) - 1$, where $B^{(l+1)} = B^{(l)} \cup \{\mathbf{b}_i \oplus \mathbf{b}_j\}$.*

According to the Corollary 1, the reduction in *TotalDist* can be computed by the supports of each target vector. We define the frequency of a pair $\mathbf{b}_i, \mathbf{b}_j$ as the number of target vectors in T whose supports contain this pair. Then, frequency of $\{\mathbf{b}_i, \mathbf{b}_j\}$ is the reduction in *TotalDist* after adding $\mathbf{b}_i \oplus \mathbf{b}_j$ into the current sequence.

Definition 5. (Frequency) Let $B^{(l)}$ be a set of vectors and T be a output target set. The frequency of a vector pair $\{\mathbf{b}_i, \mathbf{b}_j\}$ with respect to B and T is defined as $\text{frequency}_{B,T}(\mathbf{b}_i, \mathbf{b}_j) = \text{frequency}_{B,T}(\mathbf{b}_j, \mathbf{b}_i) = |\{\mathbf{t} \mid \{\mathbf{b}_i, \mathbf{b}_j\} \in \text{support}(B, \mathbf{t}), \mathbf{t} \in T\}|$.

Algorithm 4 is the pseudocode of the LCB-BP algorithm. Given the input generator set $B^{(l)}$ and the output target set T , the LCB-BP algorithm can compute a complete vector sum sequence $\mathcal{B}$. In each iteration of the LCB-BP algorithm, we can generate all

the minimal linear representations of each $\mathbf{t} \in T$. Then, the support of each $\mathbf{t}$ is obtained, and we can choose the pair with the highest frequency and add it to the sequence.

When multiple candidate pairs share the same highest frequency value, which corresponds to the case in the original BP algorithm where multiple pairs yield identical reductions in *TotalDist*, ties occurs. As this paper does not investigate tie-breaking techniques, we employ the simplest strategy of randomly selecting one candidate pair.

Algorithm 4 The pseudocode of the LCB-BP algorithm

Require: The input generator set $B^{(l)} = \{\mathbf{b}_0, \dots, \mathbf{b}_{l-1}\}$. The output target set $T = \{\mathbf{t}_0, \dots, \mathbf{t}_{m-1}\}$.

Ensure: A complete vector sum sequence $\mathcal{B}$.

- 1: $\mathcal{B} \leftarrow$ an empty sequence
- 2: **while** $T \not\subseteq \mathcal{B}$ **do**
- 3: $B \leftarrow B^{(l)} \cup \text{set}(\mathcal{B})$
- 4: $flag \leftarrow false$
- 5: **for** $\mathbf{t} \in T$ **do**
- 6: $\text{MinLinSet}(B, \mathbf{t}) \leftarrow \text{genMinimalLinCombs}(B, \mathbf{t})$
- 7: **if** $\exists |L| = 2$ where $L \in \text{MinLinSet}(B, \mathbf{t})$ **then** $\triangleright$ pre-emptive choice
- 8: random pick a linear representation $L = \{\mathbf{b}_i, \mathbf{b}_j\}$ in $\text{MinLinSet}(B, \mathbf{t})$
- 9: $\mathcal{B} \leftarrow \mathcal{B} \mid [\mathbf{b}_i \oplus \mathbf{b}_j]$
- 10: $flag \leftarrow true$
- 11: **else**
- 12: $\text{support}(B, \mathbf{t}) \leftarrow \{(\mathbf{b}_i, \mathbf{b}_j) \mid \exists L \in \text{MinLinSet}(B, \mathbf{t}) \text{ s.t. } \mathbf{b}_i, \mathbf{b}_j \in L\}$
- 13: **end if**
- 14: **end for**
- 15: **if** $flag$ is false **then**
- 16: $Candidates \leftarrow \{(\mathbf{b}_i, \mathbf{b}_j) \mid \text{frequency}_{B,T}(\mathbf{b}_i, \mathbf{b}_j) \geq \text{frequency}_{B,T}(\mathbf{b}_u, \mathbf{b}_v) \text{ for all } (\mathbf{b}_u, \mathbf{b}_v) \in \bigcup_{\mathbf{t} \in T} \text{support}(B, \mathbf{t})\}$
- 17: randomly select a pair $(\mathbf{b}_i, \mathbf{b}_j)$ from $Candidates$
- 18: $\mathcal{B} \leftarrow \mathcal{B} \mid [\mathbf{b}_i \oplus \mathbf{b}_j]$
- 19: **end if**
- 20: **end while**
- 21: **return** $\mathcal{B}$

Compared with the original BP algorithm [BP10], the LCB-BP algorithm is more efficient. Besides, the LCB-BP algorithm is at least as powerful as the original BP algorithm. Specifically, when executed multiple times, the shortest sequence length obtained by LCB-BP is guaranteed not to exceed that produced by the original BP algorithm.

The Paar algorithm[Paa97] is a special case of LCB-BP algorithm If for each $\mathbf{t} \in T$, only one linear representation is retained in the LCB-BP algorithm, then the LCB-BP algorithm becomes equivalent to the Paar algorithm [Paa97].

In the Paar algorithm, each target vector is expressed as a sum of certain vectors $\mathbf{b}_i$ from the input generator set B initially. Then in each iteration step, the Paar algorithm computes the occurrence of all possible pairs $\{\mathbf{b}_i, \mathbf{b}_j\}$ where $\mathbf{b}_i, \mathbf{b}_j \in B$. The most frequently occurring pair $\{\mathbf{b}_i, \mathbf{b}_j\}$ is selected and the $\mathbf{b}_i \oplus \mathbf{b}_j$ in linear combination is replaced by a new element $\mathbf{b}_k = \mathbf{b}_i \oplus \mathbf{b}_j$. This process repeats until all pairs occur exactly once.

The difference between the two algorithms can be explained from the perspective of linear representations. The Paar algorithm operates by maintaining a single matrix and selecting the most frequent pair $(\mathbf{b}_i, \mathbf{b}_j)$ to generate a new vector $\mathbf{b}_k = \mathbf{b}_i + \mathbf{b}_j$, then expanding the matrix by adding a new column. Each row in this matrix corresponds to one linear representation of a target vector over the set of generated vectors

$\mathbf{b}_0, \dots, \mathbf{b}_k$. Differing from the Paar algorithm, LCB-BP computes all shortest linear representations for each target vector and then computes the supports. E.g., a target vector $\mathbf{t}$ has two shortest linear representations: $\mathbf{b}_0 + \mathbf{b}_1 + \mathbf{b}_2$ and $\mathbf{b}_0 + \mathbf{b}_3 + \mathbf{b}_4$, then its support is $\{(\mathbf{b}_0, \mathbf{b}_1), (\mathbf{b}_0, \mathbf{b}_2), (\mathbf{b}_1, \mathbf{b}_2), (\mathbf{b}_0, \mathbf{b}_3), (\mathbf{b}_0, \mathbf{b}_4), (\mathbf{b}_3, \mathbf{b}_4)\}$. While in the Paar algorithm, since it only considers one linear representation, e.g., $\mathbf{t} = \mathbf{b}_0 + \mathbf{b}_1 + \mathbf{b}_2$, there are only three candidate pairs, i.e., $(\mathbf{b}_0, \mathbf{b}_1), (\mathbf{b}_0, \mathbf{b}_2), (\mathbf{b}_1, \mathbf{b}_2)$.

This implies that during each iteration, the set of candidate pairs considered by the Paar algorithm is a subset of those considered by LCB-BP algorithm. Consequently, the Paar algorithm can be viewed as a special case of the LCB-BP algorithm, and the complete sequence $\mathcal{B}$ computed by LCB-BP algorithm is guaranteed to be superior to or at least as short as that obtained by the Paar algorithm.

A Small Example Using the LCB-BP algorithm To illustrate the concepts and methods, we use the following toy example.

Given a matrix $M_{4 \times 8} = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$, the target vectors correspond-

ing to the rows of $M_{4 \times 8}$ are $\mathbf{t}_0 = (0, 1, 1, 1, 1, 0, 1, 0)$, $\mathbf{t}_1 = (1, 0, 0, 1, 1, 1, 1, 0)$, $\mathbf{t}_2 = (1, 0, 1, 0, 0, 1, 1, 1)$ and $\mathbf{t}_3 = (1, 1, 1, 0, 1, 0, 0, 1)$. The output target set is $T = \{\mathbf{t}_0, \mathbf{t}_1, \mathbf{t}_2, \mathbf{t}_3\}$. The input generator set is $B^{(8)} = \{\mathbf{b}_0, \dots, \mathbf{b}_7\}$ where $\mathbf{b}_i = \mathbf{e}_i$ for $i = 0, \dots, 7$. We show how to compute a complete vector sum sequence $\mathcal{B}^{(l)} = [\mathbf{b}_8, \dots, \mathbf{b}_{l+7}]$ via the LCB-BP algorithm.

The initial minimal linear representations for $\mathbf{t}_i$'s are as follows: $\mathbf{t}_0 = \mathbf{b}_1 \oplus \mathbf{b}_2 \oplus \mathbf{b}_3 \oplus \mathbf{b}_4 \oplus \mathbf{b}_6$, $\mathbf{t}_1 = \mathbf{b}_0 \oplus \mathbf{b}_3 \oplus \mathbf{b}_4 \oplus \mathbf{b}_5 \oplus \mathbf{b}_6$, $\mathbf{t}_2 = \mathbf{b}_0 \oplus \mathbf{b}_2 \oplus \mathbf{b}_5 \oplus \mathbf{b}_6 \oplus \mathbf{b}_7$, $\mathbf{t}_3 = \mathbf{b}_0 \oplus \mathbf{b}_1 \oplus \mathbf{b}_2 \oplus \mathbf{b}_4 \oplus \mathbf{b}_7$. Thus, we have $\text{MinLinSet}(B^{(8)}, \mathbf{t}_0) = \{\mathbf{b}_1, \mathbf{b}_2, \mathbf{b}_3, \mathbf{b}_4, \mathbf{b}_6\}$, $\text{MinLinSet}(B^{(8)}, \mathbf{t}_1) = \{\mathbf{b}_0, \mathbf{b}_3, \mathbf{b}_4, \mathbf{b}_5, \mathbf{b}_6\}$, $\text{MinLinSet}(B^{(8)}, \mathbf{t}_2) = \{\mathbf{b}_0, \mathbf{b}_2, \mathbf{b}_5, \mathbf{b}_6, \mathbf{b}_7\}$, $\text{MinLinSet}(B^{(8)}, \mathbf{t}_3) = \{\mathbf{b}_0, \mathbf{b}_1, \mathbf{b}_2, \mathbf{b}_4, \mathbf{b}_7\}$.

Then support of each $\mathbf{t}_i$ with respect to $B^{(8)}$ is as follows:

$\text{support}(B^{(8)}, \mathbf{t}_0) = \{\{\mathbf{b}_1, \mathbf{b}_2\}, \{\mathbf{b}_1, \mathbf{b}_3\}, \{\mathbf{b}_1, \mathbf{b}_4\}, \{\mathbf{b}_1, \mathbf{b}_6\}, \{\mathbf{b}_2, \mathbf{b}_3\}, \{\mathbf{b}_2, \mathbf{b}_4\}, \{\mathbf{b}_2, \mathbf{b}_6\}, \{\mathbf{b}_3, \mathbf{b}_4\}, \{\mathbf{b}_3, \mathbf{b}_6\}, \{\mathbf{b}_4, \mathbf{b}_6\}\}$,

$\text{support}(B^{(8)}, \mathbf{t}_1) = \{\{\mathbf{b}_0, \mathbf{b}_3\}, \{\mathbf{b}_0, \mathbf{b}_4\}, \{\mathbf{b}_0, \mathbf{b}_5\}, \{\mathbf{b}_0, \mathbf{b}_6\}, \{\mathbf{b}_3, \mathbf{b}_4\}, \{\mathbf{b}_3, \mathbf{b}_5\}, \{\mathbf{b}_3, \mathbf{b}_6\}, \{\mathbf{b}_4, \mathbf{b}_5\}, \{\mathbf{b}_4, \mathbf{b}_6\}, \{\mathbf{b}_5, \mathbf{b}_6\}\}$,

$\text{support}(B^{(8)}, \mathbf{t}_2) = \{\{\mathbf{b}_0, \mathbf{b}_2\}, \{\mathbf{b}_0, \mathbf{b}_5\}, \{\mathbf{b}_0, \mathbf{b}_6\}, \{\mathbf{b}_0, \mathbf{b}_7\}, \{\mathbf{b}_2, \mathbf{b}_5\}, \{\mathbf{b}_2, \mathbf{b}_6\}, \{\mathbf{b}_2, \mathbf{b}_7\}, \{\mathbf{b}_5, \mathbf{b}_6\}, \{\mathbf{b}_5, \mathbf{b}_7\}, \{\mathbf{b}_6, \mathbf{b}_7\}\}$,

$\text{support}(B^{(8)}, \mathbf{t}_3) = \{\{\mathbf{b}_0, \mathbf{b}_1\}, \{\mathbf{b}_0, \mathbf{b}_2\}, \{\mathbf{b}_0, \mathbf{b}_4\}, \{\mathbf{b}_0, \mathbf{b}_7\}, \{\mathbf{b}_1, \mathbf{b}_2\}, \{\mathbf{b}_1, \mathbf{b}_4\}, \{\mathbf{b}_1, \mathbf{b}_7\}, \{\mathbf{b}_2, \mathbf{b}_4\}, \{\mathbf{b}_2, \mathbf{b}_7\}, \{\mathbf{b}_4, \mathbf{b}_7\}\}$.

Then, frequency of each pair in these supports are computed according to Definition 5. For example, $\text{frequency}_{B^{(8)}, T}(\mathbf{b}_1, \mathbf{b}_2)$ is 2 because the pair $\{\mathbf{b}_1, \mathbf{b}_2\}$ appears in $\text{support}(B^{(8)}, \mathbf{t}_0)$ and $\text{support}(B^{(8)}, \mathbf{t}_3)$. The pairs with the highest frequency are the candidates. In this step, the highest frequency is 2 and the candidates are $\{\mathbf{b}_0, \mathbf{b}_4\}, \{\mathbf{b}_3, \mathbf{b}_4\}, \{\mathbf{b}_4, \mathbf{b}_6\}, \{\mathbf{b}_0, \mathbf{b}_2\}, \{\mathbf{b}_0, \mathbf{b}_5\}, \{\mathbf{b}_5, \mathbf{b}_6\}, \{\mathbf{b}_3, \mathbf{b}_6\}, \{\mathbf{b}_0, \mathbf{b}_7\}, \{\mathbf{b}_2, \mathbf{b}_4\}, \{\mathbf{b}_1, \mathbf{b}_2\}, \{\mathbf{b}_2, \mathbf{b}_7\}, \{\mathbf{b}_1, \mathbf{b}_4\}, \{\mathbf{b}_0, \mathbf{b}_6\}, \{\mathbf{b}_2, \mathbf{b}_6\}$. Then randomly pick one pair from the candidates.

In step 1, we choose $\{\mathbf{b}_0, \mathbf{b}_4\}$ and add $\mathbf{b}_8 = \mathbf{b}_0 \oplus \mathbf{b}_4$ into $B^{(8)}$. It becomes that $B^{(9)} = \{\mathbf{b}_0, \dots, \mathbf{b}_8\}$ and sequence $\mathcal{B} = [\mathbf{b}_8]$.

In step 2, the minimal linear representations for $\mathbf{t}_1$ and $\mathbf{t}_3$ with respect to $B^{(9)}$ become as follow:

$\mathbf{t}_1 = \mathbf{b}_3 \oplus \mathbf{b}_5 \oplus \mathbf{b}_6 \oplus \mathbf{b}_8$,

$\mathbf{t}_3 = \mathbf{b}_1 \oplus \mathbf{b}_2 \oplus \mathbf{b}_7 \oplus \mathbf{b}_8$.

Supports of $\mathbf{t}_1$ and $\mathbf{t}_3$ are $\text{support}(B^{(9)}, \mathbf{t}_1) = \{\{\mathbf{b}_3, \mathbf{b}_5\}, \{\mathbf{b}_3, \mathbf{b}_6\}, \{\mathbf{b}_3, \mathbf{b}_8\}, \{\mathbf{b}_5, \mathbf{b}_6\}, \{\mathbf{b}_5, \mathbf{b}_8\}, \{\mathbf{b}_6, \mathbf{b}_8\}\}$ and $\text{support}(B^{(9)}, \mathbf{t}_3) = \{\{\mathbf{b}_1, \mathbf{b}_2\}, \{\mathbf{b}_1, \mathbf{b}_7\}, \{\mathbf{b}_1, \mathbf{b}_8\}, \{\mathbf{b}_2, \mathbf{b}_7\}, \{\mathbf{b}_2, \mathbf{b}_8\}, \{\mathbf{b}_7, \mathbf{b}_8\}\}$, while $\text{support}(B^{(9)}, \mathbf{t}_i) = \text{support}(B^{(8)}, \mathbf{t}_i)$ for $i = 0, 2$.

The pairs with the highest frequency 2 are $\{\mathbf{b}_5, \mathbf{b}_6\}, \{\mathbf{b}_3, \mathbf{b}_6\}, \{\mathbf{b}_1, \mathbf{b}_2\}, \{\mathbf{b}_2, \mathbf{b}_7\}, \{\mathbf{b}_2, \mathbf{b}_6\}$, so we choose $\{\mathbf{b}_5, \mathbf{b}_6\}$ and we have $\mathbf{b}_9 = \mathbf{b}_5 \oplus \mathbf{b}_6$, $B^{(10)} = \{\mathbf{b}_0, \dots, \mathbf{b}_9\}$ and $\mathcal{B} = [\mathbf{b}_8, \mathbf{b}_9]$.

Step 3: $\mathbf{b}_{10} = \mathbf{b}_1 \oplus \mathbf{b}_2$. (Minimal linear representations: $\mathbf{t}_0 = \mathbf{b}_1 \oplus \mathbf{b}_2 \oplus \mathbf{b}_3 \oplus \mathbf{b}_4 \oplus \mathbf{b}_6$, $\mathbf{t}_1 = \mathbf{b}_3 \oplus \mathbf{b}_8 \oplus \mathbf{b}_9$, $\mathbf{t}_2 = \mathbf{b}_0 \oplus \mathbf{b}_2 \oplus \mathbf{b}_7 \oplus \mathbf{b}_9$, $\mathbf{t}_3 = \mathbf{b}_1 \oplus \mathbf{b}_2 \oplus \mathbf{b}_7 \oplus \mathbf{b}_8$. Candidates are $\{\mathbf{b}_1, \mathbf{b}_2\}, \{\mathbf{b}_2, \mathbf{b}_7\}$ and highest frequency is 2. Randomly pick one pair from $\{\mathbf{b}_1, \mathbf{b}_2\}$ and $\{\mathbf{b}_2, \mathbf{b}_7\}$.)

Step 4: $\mathbf{b}_{11} = \mathbf{b}_4 \oplus \mathbf{b}_{10}$. (Minimal linear representations: $\mathbf{t}_0 = \mathbf{b}_3 \oplus \mathbf{b}_4 \oplus \mathbf{b}_6 \oplus \mathbf{b}_{10}$, $\mathbf{t}_1 = \mathbf{b}_3 \oplus \mathbf{b}_8 \oplus \mathbf{b}_9$, $\mathbf{t}_2 = \mathbf{b}_0 \oplus \mathbf{b}_2 \oplus \mathbf{b}_7 \oplus \mathbf{b}_9$, $\mathbf{t}_3 = \mathbf{b}_7 \oplus \mathbf{b}_8 \oplus \mathbf{b}_{10}$. Candidates are all pairs in supports since frequency of any pair is 1. Randomly pick one pair from candidates in this step.)

Step 5: $\mathbf{b}_{12} = \mathbf{b}_0 \oplus \mathbf{b}_7$. (Minimal linear representations: $\mathbf{t}_0 = \mathbf{b}_3 \oplus \mathbf{b}_6 \oplus \mathbf{b}_{11}$, $\mathbf{t}_1 = \mathbf{b}_3 \oplus \mathbf{b}_8 \oplus \mathbf{b}_9$, $\mathbf{t}_2 = \mathbf{b}_0 \oplus \mathbf{b}_2 \oplus \mathbf{b}_7 \oplus \mathbf{b}_9$, $\mathbf{t}_3 = \mathbf{b}_7 \oplus \mathbf{b}_8 \oplus \mathbf{b}_{10} = \mathbf{b}_0 \oplus \mathbf{b}_7 \oplus \mathbf{b}_{11}$. There are two minimal linear representations of $\mathbf{t}_3$. Then $\text{support}(B^{(12)}, \mathbf{t}_3) = \{\{\mathbf{b}_7, \mathbf{b}_8\}, \{\mathbf{b}_7, \mathbf{b}_{10}\}, \{\mathbf{b}_8, \mathbf{b}_{10}\}, \{\mathbf{b}_0, \mathbf{b}_7\}, \{\mathbf{b}_0, \mathbf{b}_{11}\}, \{\mathbf{b}_7, \mathbf{b}_{11}\}\}$. There are only one pair $\{\mathbf{b}_0, \mathbf{b}_7\}$ has frequency 2 while others have frequency 1 so $\{\mathbf{b}_0, \mathbf{b}_7\}$ is selected in this step.)

Step 6: $\mathbf{b}_{13} = \mathbf{b}_{11} \oplus \mathbf{b}_{12}$. (Found $\mathbf{t}_3 = \mathbf{b}_{11} \oplus \mathbf{b}_{12}$.)

Step 7: $\mathbf{b}_{14} = \mathbf{b}_3 \oplus \mathbf{b}_6$. (Candidates are all pairs in supports since frequency of any pair is 1 and randomly pick one pair $\{\mathbf{b}_3, \mathbf{b}_6\}$.)

Step 8: $\mathbf{b}_{15} = \mathbf{b}_{11} \oplus \mathbf{b}_{14}$. (Found $\mathbf{t}_0 = \mathbf{b}_{11} \oplus \mathbf{b}_{14}$.)

Step 9: $\mathbf{b}_{16} = \mathbf{b}_5 \oplus \mathbf{b}_8$. (Minimal linear representations of $\mathbf{t}_1$ and $\mathbf{t}_2$ are: $\mathbf{t}_1 = \mathbf{b}_5 \oplus \mathbf{b}_8 \oplus \mathbf{b}_{14} = \mathbf{b}_3 \oplus \mathbf{b}_8 \oplus \mathbf{b}_9$, $\mathbf{t}_2 = \mathbf{b}_2 \oplus \mathbf{b}_9 \oplus \mathbf{b}_{12}$. $\text{support}(B^{(16)}, \mathbf{t}_1) = \{\{\mathbf{b}_5, \mathbf{b}_8\}, \{\mathbf{b}_5, \mathbf{b}_{14}\}, \{\mathbf{b}_8, \mathbf{b}_{14}\}, \{\mathbf{b}_3, \mathbf{b}_8\}, \{\mathbf{b}_3, \mathbf{b}_9\}, \{\mathbf{b}_8, \mathbf{b}_9\}\}$, $\text{support}(B^{(16)}, \mathbf{t}_2) = \{\{\mathbf{b}_2, \mathbf{b}_9\}, \{\mathbf{b}_2, \mathbf{b}_{12}\}, \{\mathbf{b}_9, \mathbf{b}_{12}\}\}$. Although there are two minimal linear representations of $\mathbf{t}_1$, the highest frequency of each pair is still 1. Randomly pick the pair $\{\mathbf{b}_5, \mathbf{b}_8\}$.)

Step 10: $\mathbf{b}_{17} = \mathbf{b}_{14} \oplus \mathbf{b}_{16}$. (Found $\mathbf{t}_1 = \mathbf{b}_{14} \oplus \mathbf{b}_{16}$.)

Step 11: $\mathbf{b}_{18} = \mathbf{b}_2 \oplus \mathbf{b}_9$. (Randomly pick one pair $\{\mathbf{b}_2, \mathbf{b}_9\}$ from the candidates $\{\{\mathbf{b}_2, \mathbf{b}_9\}, \{\mathbf{b}_9, \mathbf{b}_{12}\}, \{\mathbf{b}_2, \mathbf{b}_{12}\}\}$ since the frequency of each pair is 1.)

Step 12: $\mathbf{b}_{19} = \mathbf{b}_{12} \oplus \mathbf{b}_{18}$. (Found $\mathbf{t}_2 = \mathbf{b}_{12} \oplus \mathbf{b}_{18}$.)

Finally, we compute the complete vector sum sequence $\mathcal{B}^{(12)} = [\mathbf{b}_8, \dots, \mathbf{b}_{19}]$ and $\mathbf{t}_0 = \mathbf{b}_{15}$, $\mathbf{t}_1 = \mathbf{b}_{17}$, $\mathbf{t}_2 = \mathbf{b}_{19}$, $\mathbf{t}_3 = \mathbf{b}_{13}$.

3.3 The MILP model for generating minimal linear combinations

In the LCB-BP algorithm, we compute all the linear representations by constructing the MILP model instead of calling the `reachable()` function iteratively. The pseudocode of the MILP model for generating minimal linear representations is given in Algorithm 5. During implementation, the minimal linear representations can partially be obtained directly from the results of previous steps. Therefore, we only need to use the MILP model to generate combinations that cannot be derived in this way.

4 A new strategy for local optimization

Since it is difficult to compute the global optimal sequence using the BP algorithm, the general method is to compute a sufficiently good sequence first, then perform local optimization on certain parts of the sequence. In this section, we introduce concepts of graph representation in subsection 4.1 first. Then we give a new strategy for selecting a subsequence for local optimization in subsection 4.2.

Algorithm 5 An MILP model for generating minimal linear combinations**Require:** The set of vectors $B = \{\mathbf{b}_0, \dots, \mathbf{b}_{l-1}\}$, $\mathbf{b}_i \in \mathbb{F}_2^n$. The target vector $\mathbf{t} \in \mathbb{F}_2^n$.**Ensure:** The set of minimal linear representations MinLinSet.

```

1: Prepare an empty MILP Model  $\mathcal{M}$ 
2:  $\mathcal{M}.var \leftarrow c[i]$  as binary variables for  $0 \leq i < l$ 
3: for  $j = 0$  to  $n - 1$  do
4:    $\mathcal{M}.con \leftarrow N$  as integer variable range between 0 and  $l$ 
5:    $\mathcal{M}.con \leftarrow \sum_{i=0}^{l-1} c[i] \cdot \mathbf{b}_i[j] + \mathbf{t}[j] = 2 \cdot N \quad \triangleright \mathbf{b}_i[j]$  and  $\mathbf{t}[j]$  are the  $j$ -th bit in  $\mathbf{b}_i$ 
     and  $\mathbf{t}$ 
6: end for
7:  $\mathcal{M}.obj \leftarrow \min \sum_{i=0}^{l-1} c[i]$ 
8:  $\mathcal{M}.optimize()$ 
9:  $numSol \leftarrow$  the number of optimal solutions
10: for  $s = 0$  to  $numSol - 1$  do
11:    $\mathcal{M}.solutionNumber \leftarrow s$ 
12:   MinLinSet  $\leftarrow$  MinLinSet  $\cup \{\mathbf{b}_i \mid c[i] = 1\}$ 
13: end for
14: return MinLinSet

```

4.1 The graph representation for sequences

In existing strategy for selecting subsequences, only consecutive elements were considered. However, rearranging the order of certain elements within the complete vector sum sequence can also yield a new sequence which is a complete vector sum sequence as well. Therefore, only considering consecutive elements is insufficient. To characterize the relationship between sequences differing only in element ordering, we propose a *graph representation*.

In graph representation, each element in vector sum sequence corresponds to a *vertex* in the graph. *Edges* indicate generation relationships (e.g., $\mathbf{b}_i \rightarrow \mathbf{b}_k$ and $\mathbf{b}_j \rightarrow \mathbf{b}_k$ denotes $\mathbf{b}_k = \mathbf{b}_i \oplus \mathbf{b}_j$). Vectors in input generator set have *in-degree* 0, the other vertices have *in-degree* 2 since each basis vector is generated from exactly two predecessors: $\mathbf{b}_k = \mathbf{b}_i \oplus \mathbf{b}_j$.

For example, $\mathcal{B}^{15} = [\mathbf{b}_8, \dots, \mathbf{b}_{22}]$ is a complete vector sum sequence for $T = \{\mathbf{t}_0, \dots, \mathbf{t}_3\}$ with respect to $B = \{\mathbf{e}_0, \dots, \mathbf{e}_7\}$. Figure 1 shows the sequence and its graph representation. The blue nodes are the vectors in input generator set B and the red nodes are the vectors in output target set T , where $\mathbf{b}_i = \mathbf{e}_i$ for $i = 0, \dots, 7$ and $\mathbf{t}_0 = \mathbf{b}_{15}$, $\mathbf{t}_1 = \mathbf{b}_{20}$, $\mathbf{t}_2 = \mathbf{b}_{21}$, $\mathbf{t}_3 = \mathbf{b}_{22}$. The gray nodes ($\mathbf{b}_i$ for $i = 10, 12, 13, 14, 16, 17, 18, 19$) have out-degree 1 while other nodes ($\mathbf{b}_8, \mathbf{b}_9, \mathbf{b}_{11}$) are green.

4.2 A new strategy for selecting subgraph

Our experimental results revealed that a significant portion of randomly selected subsequences could not be further optimized. In contrast, those subsequences that can be replaced by a shorter one always consist of vectors with an out-degree of 1. The out-degree of a node $\mathbf{b}_i$ reflects the number of vectors directly generated from $\mathbf{b}_i$. A higher out-degree indicates that more vectors are generated from $\mathbf{b}_i$, which means $\mathbf{b}_i$ has greater importance to the current sequence, making it harder to be replaced. Conversely, the subgraph constructed by nodes with out-degree 1 is easier to replace with fewer nodes. Previous approaches employed a random strategy, resulting in extensive reconstruction efforts, even for subgraphs that are unlikely to be further optimized. Our strategy avoids some unnecessary computation.

Based on these observations, we propose a new strategy to select subsequences for local optimization. Let $B^{(l)}$ be a input generator set and T be a output target set. Given

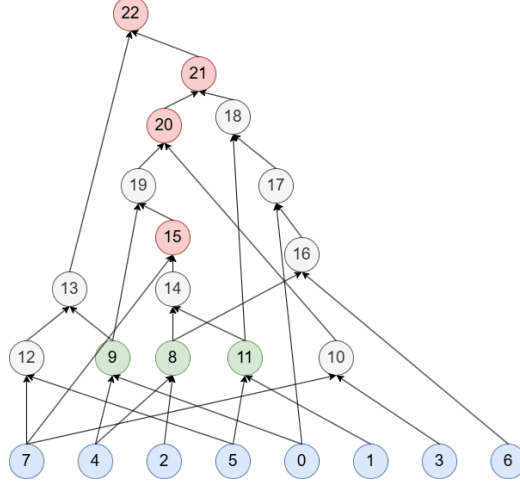


Figure 1: The graph representation of sequence $\mathcal{B}^{(15)}$.

a complete vector sum sequence $\mathcal{B}^{(s)}$ for T with respect to $B^{(l)}$. Here are the process for local optimization:

- Step 1 Compute the out-degree of each node in graph of $\mathcal{B}^{(s)}$ and record nodes with out-degree 1 which are not target nodes. Denote this set as $B_{\text{out}1}$.
- Step 2 Randomly select some nodes from $B_{\text{out}1}$ and denote these nodes as D , i.e. $D \subset B_{\text{out}1}$. For each $\mathbf{b} \in D$, find its parent nodes as *sub-target nodes*. Denote the sub-target point set as T_{sub} .
- Step 3 Compute a vector sum sequence $\bar{\mathcal{B}}^{(u)}$ with respect to $B^{(l)}$ consist of u vectors in $B^{(s)} \setminus D$, where $B^{(s)} = \text{set}(\mathcal{B}^{(s)})$ and the value of u should be maximized.
- Step 4 Construct a new SLP problem, where input generator set is $\bar{B}^{(u)} = B^{(l)} \cup \text{set}(\bar{\mathcal{B}}^{(u)})$ and output target set is T_{sub} and solve this subproblem. Let $\bar{\mathcal{B}}^{(v)}$ be a complete vector sum sequence for T_{sub} with respect to $\bar{B}^{(u)}$, then we can use the vectors in $\bar{\mathcal{B}}^{(v)}$ to replace the set D . If $v < |D|$, then a shorter complete vector sum sequence is obtained; otherwise, go to Step 2.

Algorithm 6 is used to generate the vector sum sequence $\bar{\mathcal{B}}^{(u)}$ in Step 3. After generating a sequence $\bar{\mathcal{B}}^{(v)}$ we can use the algorithm 6 to generate a sequence from the set of vectors $B^{(s)} \setminus D \cup \text{set}(\bar{\mathcal{B}}^{(v)})$ as well.

Methods in subsection 3.1 and 3.2 can be used to solve the subproblem in Step 4. If the number of elements in D is relatively small, and the number of new vectors to be generated in Step 4 is also limited, then the problem becomes relatively straightforward. In such cases, the MILP model described in subsection 3.1 can be applied for local optimization. However, if the MILP model cannot solve within a reasonable time, heuristic methods can also be used for local optimization.

By employing this strategy, we can simplify the sequence $\mathcal{B}^{(15)}$ shown in Figure 1. First, we select three nodes to form the set $D = \{\mathbf{b}_{13}, \mathbf{b}_{14}, \mathbf{b}_{19}\}$. All of these nodes have an out-degree of 1 and are not target nodes. The corresponding set of sub-target vectors is then defined as $T_{\text{sub}} = \{\mathbf{b}_{15}, \mathbf{b}_{20}, \mathbf{b}_{22}\}$. After computing the generating set $\bar{B}^{(16)} = \{\mathbf{b}_0, \dots, \mathbf{b}_{12}, \mathbf{b}_{16}, \dots, \mathbf{b}_{18}\}$ via the Algorithm 6, we construct a subproblem aimed at finding a shorter sequence, as illustrated in Figure 2. In this subproblem, the red nodes represent vectors in the target set, while the blue nodes represent vectors in the generating set.

Algorithm 6 The pseudocode for generating the vector sum sequence $\mathcal{B}'^u$

Require: Sets of vectors $B^{(s)}$ and D . The input generator set $B^{(l)}$.

Ensure: A vector sum sequence $\tilde{\mathcal{B}}$.

```

1:  $\tilde{\mathcal{B}} \leftarrow$  an empty sequence
2:  $B \leftarrow \{\mathbf{b}_0, \mathbf{b}_1, \dots, \mathbf{b}_{n-1}\}$ 
3: while True do
4:    $num \leftarrow 0$ 
5:   for  $\mathbf{b}_k \in B^{(s)} \setminus D$  do
6:     if  $\exists \mathbf{b}_i, \mathbf{b}_j \in B$  such that  $\mathbf{b}_k = \mathbf{b}_i \oplus \mathbf{b}_j$  then
7:        $\tilde{\mathcal{B}} \leftarrow \tilde{\mathcal{B}} \mid [\mathbf{b}_k]$ 
8:        $B \leftarrow B \cup \{\mathbf{b}_k\}$ 
9:        $num \leftarrow num + 1$ 
10:    end if
11:  end for
12:  if  $num = 0$  then
13:    break
14:  end if
15: end while
16: return  $\tilde{\mathcal{B}}$ 

```

As long as we can compute the new nodes to complete the graph representation of Figure 2 into a full sequence, we can obtain a new complete vector sum sequence. Figure 3 shows the new complete vector sum sequence, which consists of only 14 vectors. Here, we use two vectors (represented by the green nodes) to replace the three vectors originally in D . Our method for local optimization may yields better results than existing methods, as it can introduce new vectors into the sequence for replacement, whereas existing methods were limited to removing redundant points. This enhanced flexibility potentially leads to better local optimization outcomes.

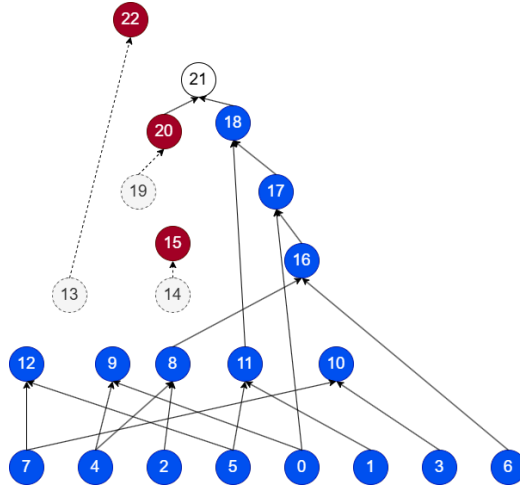


Figure 2: The subproblem for local optimization for $\mathcal{B}^{(15)}$.

When the number of nodes with out-degree 1 is relatively small, we can expand the candidate set D to include nodes with out-degree 2, thereby increasing the size of D and enhancing the effectiveness of local optimization.

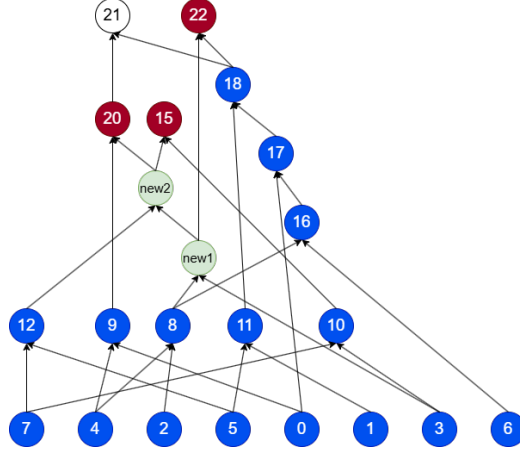


Figure 3: The graph of representation $\mathcal{B}^{(14)}$.

5 Results

In this section, we first present in Subsection 5.1 the efficiency comparison between the LCB-BP algorithm and existing BP algorithms as well as their variants. Then, in Subsection 5.2, we introduce the structural characteristics of the AES linear-layer matrix and propose algorithm optimizations based on these features, leading to new g-XOR results for AES. Finally, we apply the proposed method to other matrices and present the corresponding results in Subsection 5.3.

5.1 Efficiency Comparison between LCB-BP and BP Algorithms

We implemented the LCB-BP algorithm in C++ and compared its computational time and results with the original BP algorithm [BP18]. The experiments were conducted on a system with an AMD Ryzen 9 7950x 16-core processor with 32 CPUs and 128GB of RAM, running Ubuntu 20.04.2 LTS.

The test set consists of random square matrices categorized by their size and density. Specifically, we selected 10 matrices for each size (ranging from 15×15 to 20×20) and density ρ ranging from 0.1 to 0.9, resulting in a total of 540 matrices for our comparative analysis.

Each algorithm was applied to solve these matrices. The LCB-BP algorithm was executed 50 times for each matrix. We recorded the average computation time over 50 runs. Since the original BP algorithm produces deterministic outputs, it was executed only once per matrix. We also recorded the number of instances where BP and LCB-BP algorithms achieved the lowest g-XOR counts.

Each algorithm was applied to solve these matrices. The LCB-BP algorithm was executed 50 times for each matrix, while the original BP algorithm, being deterministic, was executed only once per matrix. We recorded the average computation time over the 50 runs for LCB-BP and the single run time for BP. Additionally, we counted the number of instances where each algorithm achieved the lowest g-XOR counts among all compared algorithms.

For example, for a 15×15 matrix with density $\rho = 0.1$, if the minimum g-XOR value obtained by any algorithm is 16, and the BP algorithm also yields 16, then $Res_{BP,15,0.1} = 1$; otherwise, $Res_{BP,15,0.1} = 0$. For 15×15 matrices, which include 10 matrices at each of the 9 density levels (90 matrices total), the total number of instances where BP achieved the lowest g-XOR counts is calculated as $Res_{BP,15} = \sum_{d \in \{0.1, \dots, 0.9\}} Res_{BP,15,d}$.

Table 1: Experimental results comparison between BP and LCB-BP algorithms

Matrix Size	Algorithm	#Lowest Instance	Avg. Time (s)
15 × 15	BP	33	0.82
15 × 15	LCB-BP	87	0.28
16 × 16	BP	44	2.72
16 × 16	LCB-BP	79	0.35
17 × 17	BP	35	7.63
17 × 17	LCB-BP	78	0.41
18 × 18	BP	52	24.28
18 × 18	LCB-BP	73	0.53
19 × 19	BP	41	71.92
19 × 19	LCB-BP	76	0.67
20 × 20	BP	53	235.99
20 × 20	LCB-BP	68	0.85

The statistical results are summarized in the below. Based on the experimental results presented in Table 1. The column “#Lowest Instance” in Table 1 represents the value of $Res_{BP,m}$ for different matrix sizes $m \times m$, indicating the number of matrices of a given size for which each algorithm achieved the lowest g-XOR count.

Several observations can be drawn regarding the performance of the BP and LCB-BP algorithms. First, in terms of solution quality, the LCB-BP algorithm demonstrates superior capability in finding optimal implementations compared to the original BP algorithm. Across all matrix sizes from 15×15 to 20×20 , the LCB-BP algorithm consistently achieves a higher number of lowest g-XOR count instances. More notably, the computational efficiency of LCB-BP shows remarkable improvement over the BP algorithm. The average computation time for LCB-BP remains substantially lower across all matrix sizes. The experimental results clearly demonstrate that the LCB-BP algorithm not only produces higher-quality solutions but also achieves this with significantly reduced computational overhead.

5.2 Results of AES MixColumns matrix

Observations on the structure of AES MixColumns matrix[DR02] The MixColumns operation is a fundamental linear transformation within the AES, applied to each column of the cipher state. It is a circulant MDS (Maximum Distance Separable) matrix. The AES MixColumns matrix can be written as a $M_{32 \times 32}$ over $\mathbb{F}_2$, as illustrated in Table 2. We rearranged the order of the rows in AES MixColumns matrix and grouped the reordered rows, as illustrated in Table 3. We observe that the linear representations of row vectors within the same group all match the same pattern. Here we present eight patterns to describe the vectors of rows in the different groups.

Let $I_i = \{\mathbf{e}_j \mid i = j \pmod{8}, 0 \leq j < 32\}$, i.e. $I_0 = \{\mathbf{e}_0, \mathbf{e}_8, \mathbf{e}_{16}, \mathbf{e}_{24}\}, \dots, I_7 = \{\mathbf{e}_7, \mathbf{e}_{15}, \mathbf{e}_{23}, \mathbf{e}_{31}\}$. Let T_{AES} be the set of target vectors of the AES MixColumns matrix. As illustrated in Figure 3, $\mathbf{t} \in T_{AES}$ follows either of the eight patterns, where L_i is the linear representation of the target vector $\mathbf{t}_i$ and $\mathbf{t}_i$ corresponds to the i -th row of the AES MixColumns matrix:

Pattern-0: $L_{8 \times i+0} = I_{0,i} \cup I_{7,i}$, where $I_{0,i} \subseteq I_0, I_{7,i} \subseteq I_7, 0 \leq i < 3$.

Pattern-1: $L_{8 \times i+1} = I_{0,i} \cup I_{1,i} \cup I_{7,i}$, where $I_{0,i} \subseteq I_0, I_{1,i} \subseteq I_1, I_{7,i} \subseteq I_7, 0 \leq i < 3$.

Pattern-2: $L_{8 \times i+2} = I_{1,i} \cup I_{2,i}$, where $I_{1,i} \subseteq I_1, I_{2,i} \subseteq I_2, 0 \leq i < 3$.

Pattern-3: $L_{8 \times i+3} = I_{2,i} \cup I_{3,i} \cup I_{7,i}$, where $I_{2,i} \subseteq I_2, I_{3,i} \subseteq I_3, I_{7,i} \subseteq I_7, 0 \leq i < 3$.

Pattern-4: $L_{8 \times i+4} = I_{3,i} \cup I_{4,i} \cup I_{7,i}$, where $I_{3,i} \subseteq I_3, I_{4,i} \subseteq I_4, I_{7,i} \subseteq I_7, 0 \leq i < 3$.

Pattern-5: $L_{8 \times i+5} = I_{4,i} \cup I_{5,i}$, where $I_{4,i} \subseteq I_4, I_{5,i} \subseteq I_5, 0 \leq i < 3$.

[illegible]

Table 3: A row-permuted matrix of the AES MixColumns.

t_0	1	1	1	1	1	1
t_8	1	1	1	1	1	1
t_{16}	1	1	1	1	1	1
t_{24}	1	1	1	1	1	1
t_1	1	1	1	1	1	1
t_9	1	1	1	1	1	1
t_{17}	1	1	1	1	1	1
t_{25}	1	1	1	1	1	1
t_2	1	1	1	1	1	1
t_{10}	1	1	1	1	1	1
t_{18}	1	1	1	1	1	1
t_{26}	1	1	1	1	1	1
t_3	1	1	1	1	1	1
t_{11}	1	1	1	1	1	1
t_{19}	1	1	1	1	1	1
t_{27}	1	1	1	1	1	1
t_4	1	1	1	1	1	1
t_{12}	1	1	1	1	1	1
t_{20}	1	1	1	1	1	1
t_{28}	1	1	1	1	1	1
t_5	1	1	1	1	1	1
t_{13}	1	1	1	1	1	1
t_{21}	1	1	1	1	1	1
t_{29}	1	1	1	1	1	1
t_6	1	1	1	1	1	1
t_{14}	1	1	1	1	1	1
t_{22}	1	1	1	1	1	1
t_{30}	1	1	1	1	1	1
t_7	1	1	1	1	1	1
t_{15}	1	1	1	1	1	1
t_{23}	1	1	1	1	1	1
t_{31}	1	1	1	1	1	1

If the linear representation of a vector with respect to the unit vectors matches Pattern- i , then we say that the vector matches Pattern- i . The target vectors matching the exact

pattern share many common positions where the entry is 1.

By leveraging the patterns, we can improve the local optimization stage. We observe the results in Lin et al. [LXZZ21] and find that all the vectors in their sequences match the patterns. Furthermore, for any target vector $\mathbf{t}$ that matches Pattern-i, all vectors involved in generating $\mathbf{t}$ in the sequence of [LXZZ21] also match Pattern-i. It is important to note that the number of vectors matching a given pattern is substantially smaller than the total number of vectors in $\mathbb{F}_2^{32}$. For example, there are only 256 vectors that match Pattern-0. Based on the observations, we can further reduce the size of the MILP model for local optimization and enhance the solving efficiency of the model. In local optimization stage, when the vectors in T_{sub} match specific patterns, it is sufficient to retain those vectors in $\bar{B}^{(u)}$ which also match these patterns. Thus, the scale of subproblem is reduced, then it significantly enhances the efficiency of this optimization stage.

This property can accelerate the local optimization method proposed in this paper. However, it may not be applicable to improve existing local optimization algorithms. The primary reason lies in the fact that current local optimization techniques can only reduce existing redundant operations to achieve lower g-XOR counts, whereas our local optimization approach can replace selected subsequences with shorter ones that may potentially include new vectors not present in the original sequence. This property reduces the search space for replacement subsequences, thereby accelerating the local optimization process. Furthermore, by leveraging the efficient LCB-BP algorithm, we can obtain more high-quality initial sequences. Consequently, we have achieved superior g-XOR results for the AES MixColumns matrix.

g-XOR Counts for the AES MixColumns Matrix We apply our methods to optimize the implementation of AES MixColumns matrix and three sequences with 89 g-XOR operations, are found. Table 4 lists one of the sequences we found.

Table 4: An implementation of AES MixColumns matrix with 89 g-XOR. $\mathbf{b}_0, \mathbf{b}_1, \dots, \mathbf{b}_{31}$ are the 32 input vectors, which are unit vectors. $\mathbf{t}_0, \mathbf{t}_1, \dots, \mathbf{t}_{31}$ are the 32 target vectors.

$\mathbf{b}_{32} = \mathbf{b}_0 \oplus \mathbf{b}_{24}$	$\mathbf{b}_{33} = \mathbf{b}_1 \oplus \mathbf{b}_9$	$\mathbf{b}_{34} = \mathbf{b}_2 \oplus \mathbf{b}_{26}$	$\mathbf{b}_{35} = \mathbf{b}_3 \oplus \mathbf{b}_{19}$	$\mathbf{b}_{36} = \mathbf{b}_3 \oplus \mathbf{b}_{27}$
$\mathbf{b}_{37} = \mathbf{b}_4 \oplus \mathbf{b}_{28}$	$\mathbf{b}_{38} = \mathbf{b}_5 \oplus \mathbf{b}_{13}$	$\mathbf{b}_{39} = \mathbf{b}_6 \oplus \mathbf{b}_{14}$	$\mathbf{b}_{40} = \mathbf{b}_6 \oplus \mathbf{b}_{22}$	$\mathbf{b}_{41} = \mathbf{b}_7 \oplus \mathbf{b}_{23}$
$\mathbf{b}_{42} = \mathbf{b}_7 \oplus \mathbf{b}_{31}$	$\mathbf{b}_{43} = \mathbf{b}_9 \oplus \mathbf{b}_{25}$	$\mathbf{b}_{44} = \mathbf{b}_{10} \oplus \mathbf{b}_{18}$	$\mathbf{b}_{45} = \mathbf{b}_{10} \oplus \mathbf{b}_{26}$	$\mathbf{b}_{46} = \mathbf{b}_{12} \oplus \mathbf{b}_{20}$
$\mathbf{b}_{47} = \mathbf{b}_{12} \oplus \mathbf{b}_{28}$	$\mathbf{b}_{48} = \mathbf{b}_{13} \oplus \mathbf{b}_{29}$	$\mathbf{b}_{49} = \mathbf{b}_{15} \oplus \mathbf{b}_{23}$	$\mathbf{b}_{50} = \mathbf{b}_{15} \oplus \mathbf{b}_{31}$	$\mathbf{b}_{51} = \mathbf{b}_{21} \oplus \mathbf{b}_{29}$
$\mathbf{b}_{52} = \mathbf{b}_{22} \oplus \mathbf{b}_{30}$	$\mathbf{b}_{53} = \mathbf{b}_0 \oplus \mathbf{b}_{41}$	$\mathbf{b}_{54} = \mathbf{b}_{16} \oplus \mathbf{b}_{32}$	$\mathbf{b}_{55} = \mathbf{b}_2 \oplus \mathbf{b}_{33}$	$\mathbf{b}_{56} = \mathbf{b}_{17} \oplus \mathbf{b}_{33}$
$\mathbf{b}_{57} = \mathbf{b}_4 \oplus \mathbf{b}_{36}$	$\mathbf{b}_{58} = \mathbf{b}_{19} \oplus \mathbf{b}_{36}$	$\mathbf{b}_{59} = \mathbf{b}_{21} \oplus \mathbf{b}_{37}$	$\mathbf{b}_{60} = \mathbf{b}_5 \oplus \mathbf{b}_{46}$	$\mathbf{b}_{61} = \mathbf{b}_{14} \oplus \mathbf{b}_{38}$
$\mathbf{b}_{62} = \mathbf{b}_{31} \oplus \mathbf{b}_{39}$	$\mathbf{b}_{63} = \mathbf{b}_{11} \oplus \mathbf{b}_{50}$	$\mathbf{b}_{64} = \mathbf{b}_{16} \oplus \mathbf{b}_{50}$	$\mathbf{b}_{65} = \mathbf{b}_{15} \oplus \mathbf{b}_{52}$	$\mathbf{b}_{66} = \mathbf{b}_{30} \oplus \mathbf{b}_{51}$
$\mathbf{b}_{67} = \mathbf{b}_{35} \oplus \mathbf{b}_{42}$	$\mathbf{b}_{68} = \mathbf{b}_{36} \oplus \mathbf{b}_{45}$	$\mathbf{b}_{69} = \mathbf{b}_{35} \oplus \mathbf{b}_{47}$	$\mathbf{b}_{70} = \mathbf{b}_{40} \oplus \mathbf{b}_{41}$	$\mathbf{b}_{71} = \mathbf{b}_{40} \oplus \mathbf{b}_{48}$
$\mathbf{b}_{72} = \mathbf{b}_{44} \oplus \mathbf{b}_{49}$	$\mathbf{b}_{73} = \mathbf{b}_{47} \oplus \mathbf{b}_{48}$	$\mathbf{b}_{74} = \mathbf{b}_{25} \oplus \mathbf{b}_{56}$	$\mathbf{b}_{75} = \mathbf{b}_{27} \oplus \mathbf{b}_{63}$	$\mathbf{b}_{76} = \mathbf{b}_{32} \oplus \mathbf{b}_{56}$
$\mathbf{b}_{77} = \mathbf{b}_{49} \oplus \mathbf{b}_{54}[\mathbf{t}_8]$	$\mathbf{b}_{78} = \mathbf{b}_{44} \oplus \mathbf{b}_{55}$	$\mathbf{b}_{79} = \mathbf{b}_{42} \oplus \mathbf{b}_{57}$	$\mathbf{b}_{80} = \mathbf{b}_{38} \oplus \mathbf{b}_{59}[\mathbf{t}_{29}]$	$\mathbf{b}_{81} = \mathbf{b}_{59} \oplus \mathbf{b}_{73}[\mathbf{t}_5]$
$\mathbf{b}_{82} = \mathbf{b}_{61} \oplus \mathbf{b}_{71}[\mathbf{t}_{30}]$	$\mathbf{b}_{83} = \mathbf{b}_{51} \oplus \mathbf{b}_{60}[\mathbf{t}_{13}]$	$\mathbf{b}_{84} = \mathbf{b}_{52} \oplus \mathbf{b}_{61}[\mathbf{t}_6]$	$\mathbf{b}_{85} = \mathbf{b}_{60} \oplus \mathbf{b}_{73}[\mathbf{t}_{21}]$	$\mathbf{b}_{86} = \mathbf{b}_{65} \oplus \mathbf{b}_{70}[\mathbf{t}_{31}]$
$\mathbf{b}_{87} = \mathbf{b}_{66} \oplus \mathbf{b}_{71}[\mathbf{t}_{14}]$	$\mathbf{b}_{88} = \mathbf{b}_{49} \oplus \mathbf{b}_{62}[\mathbf{t}_7]$	$\mathbf{b}_{89} = \mathbf{b}_{39} \oplus \mathbf{b}_{66}[\mathbf{t}_{22}]$	$\mathbf{b}_{90} = \mathbf{b}_{62} \oplus \mathbf{b}_{70}[\mathbf{t}_{15}]$	$\mathbf{b}_{91} = \mathbf{b}_{42} \oplus \mathbf{b}_{65}[\mathbf{t}_{23}]$
$\mathbf{b}_{92} = \mathbf{b}_{34} \oplus \mathbf{b}_{67}$	$\mathbf{b}_{93} = \mathbf{b}_{41} \oplus \mathbf{b}_{69}$	$\mathbf{b}_{94} = \mathbf{b}_{63} \oplus \mathbf{b}_{68}$	$\mathbf{b}_{95} = \mathbf{b}_{58} \oplus \mathbf{b}_{72}[\mathbf{t}_{11}]$	$\mathbf{b}_{96} = \mathbf{b}_{43} \oplus \mathbf{b}_{76}$
$\mathbf{b}_{97} = \mathbf{b}_{43} \oplus \mathbf{b}_{78}[\mathbf{t}_{26}]$	$\mathbf{b}_{98} = \mathbf{b}_{34} \oplus \mathbf{b}_{78}[\mathbf{t}_2]$	$\mathbf{b}_{99} = \mathbf{b}_8 \oplus \mathbf{b}_{77}$	$\mathbf{b}_{100} = \mathbf{b}_{45} \oplus \mathbf{b}_{74}$	$\mathbf{b}_{101} = \mathbf{b}_{42} \oplus \mathbf{b}_{76}[\mathbf{t}_{25}]$
$\mathbf{b}_{102} = \mathbf{b}_{11} \oplus \mathbf{b}_{92}[\mathbf{t}_{27}]$	$\mathbf{b}_{103} = \mathbf{b}_{92} \oplus \mathbf{b}_{94}[\mathbf{t}_3]$	$\mathbf{b}_{104} = \mathbf{b}_{46} \oplus \mathbf{b}_{79}[\mathbf{t}_{28}]$	$\mathbf{b}_{105} = \mathbf{b}_{72} \oplus \mathbf{b}_{94}[\mathbf{t}_{19}]$	$\mathbf{b}_{106} = \mathbf{b}_{79} \oplus \mathbf{b}_{93}[\mathbf{t}_{20}]$
$\mathbf{b}_{107} = \mathbf{b}_{64} \oplus \mathbf{b}_{99}[\mathbf{t}_{16}]$	$\mathbf{b}_{108} = \mathbf{b}_{97} \oplus \mathbf{b}_{100}[\mathbf{t}_{10}]$	$\mathbf{b}_{109} = \mathbf{b}_{55} \oplus \mathbf{b}_{100}[\mathbf{t}_{18}]$	$\mathbf{b}_{110} = \mathbf{b}_{53} \oplus \mathbf{b}_{99}[\mathbf{t}_0]$	$\mathbf{b}_{111} = \mathbf{b}_{96} \oplus \mathbf{b}_{99}[\mathbf{t}_9]$
$\mathbf{b}_{112} = \mathbf{b}_{75} \oplus \mathbf{b}_{104}$	$\mathbf{b}_{113} = \mathbf{b}_{54} \oplus \mathbf{b}_{110}$	$\mathbf{b}_{114} = \mathbf{b}_{37} \oplus \mathbf{b}_{112}[\mathbf{t}_4]$	$\mathbf{b}_{115} = \mathbf{b}_{93} \oplus \mathbf{b}_{112}[\mathbf{t}_{12}]$	$\mathbf{b}_{116} = \mathbf{b}_1 \oplus \mathbf{b}_{113}$
$\mathbf{b}_{117} = \mathbf{b}_{64} \oplus \mathbf{b}_{113}[\mathbf{t}_{24}]$	$\mathbf{b}_{118} = \mathbf{b}_{111} \oplus \mathbf{b}_{116}$	$\mathbf{b}_{119} = \mathbf{b}_{74} \oplus \mathbf{b}_{116}[\mathbf{t}_1]$	$\mathbf{b}_{120} = \mathbf{b}_{101} \oplus \mathbf{b}_{118}[\mathbf{t}_{17}]$	

5.3 Applications to Other Matrices

Our approach has been applied to both matrices employed in several ciphers and newly designed matrices, as did in [TP19, XZL⁺20, LXZZ21]. Since the solution quality of LCB-BP algorithm is no worse than the original BP algorithm, and the original BP algorithm

already achieves best results for 16×16 problems, we specifically evaluated our approach on 32×32 and 64×64 matrices. The experimental results are summarized in Table 5 and Table 6. The findings demonstrate that our method achieves superior performance on large-scale matrices, such as 64×64 instances, which can be attributed to the enhanced efficiency of our LCB-BP algorithm. In contrast, alternative methods often fail to compute a sequence lower enough within a reasonable time frame. Furthermore, for 32×32 matrices, we have improved upon previous best-known results for several cases.

Table 5: Implementation cost of cipher matrices under different algorithms.

Ciphers	Size	[KLSW17] ¹	[KLSW17] ²	[KLSW17] ³	[BF19]	[LXZZ21]	[YWS ⁺ 24] ⁴	This paper
FOX MU8[JV04]	64	611	-	594	592	-	-	530
GR0STL[GKM ⁺ 09]	64	493	-	475	460	-	-	421
WHIRLPOOL[BR03]	64	481	-	465	464	-	-	417
AES[DR02]	32	108	108	97	95	91	91	89
FOX MU4[JV04]	32	144	143	137	131	130	130	128

1 This column presents the results in [KLSW17] obtained by Paar1’s heuristic.

2 This column presents the results in [KLSW17] obtained by Paar2’s heuristic.

3 This column presents the results in [KLSW17] obtained by BP algorithm.

4 This column presents the results of s-XOR counts.

Table 6: Implementation cost of newly designed 8×8 matrices over $GL(8, \mathbb{F}_2)$.

Ciphers	[KLSW17] ¹	[KLSW17] ²	[KLSW17] ³	[BF19]	[XZL ⁺ 20] ⁴	This paper
[SKOP15]	474	-	467	460	-	443
[LS16]	464	-	447	443	-	422
[BKL16]	506	-	498	497	-	461
[SS17]	447	-	438	436	-	411
[SKOP15](Involutory)	430	-	428	419	348	387
[JPST17]	620	-	599	591	-	524

1 This column presents the results in [KLSW17] obtained by Paar1’s heuristic.

2 This column presents the results in [KLSW17] obtained by Paar2’s heuristic.

3 This column presents the results in [KLSW17] obtained by BP algorithm.

4 This column presents the results of s-XOR counts.

6 Conclusion

In conclusion, this work presents significant improvements in solving the shortest linear program problem. We observe that existing two-stage frameworks address the SLP problem in both stages. To enhance this approach, we revisit the BP algorithm and propose the LCB-BP algorithm, which demonstrates superior efficiency through its linear combination voting mechanism.

Furthermore, we introduce a novel subsequence selection strategy for local optimization and develop an MILP model to effectively solve local optimization problems. When applied to cryptographic matrices, particularly the AES MixColumns matrix, we identified specific structural characteristics and, leveraging these characteristics in conjunction with our proposed methods, achieved implementations requiring only 89 g-XOR operations, which surpasses the previous record of 91 g-XOR operations documented in [LXZZ21].

Additionally, we propose a conjecture regarding the BP algorithm: for our LCB-BP approach, which currently selects pairs based on the highest frequency within the Support set, we hypothesize that modifying the selection strategy to choose pairs directly from the Support set could potentially lead to global optimal solutions. We have established preliminary validation of this conjecture under certain conditions and intend to pursue formal proof in subsequent research.

References

- [BC20] Christina Boura and Daniel Coggia. Efficient MILP Modelings for Sboxes and Linear Layers of SPN Ciphers. *IACR Transactions on Symmetric Cryptology*, pages 327–361, 2020.
- [BFI19] Subhadeep Banik, Yuki Funabiki, and Takanori Isobe. More Results on Shortest Linear Programs. In Nuttapon Attrapadung and Takeshi Yagi, editors, *Advances in Information and Computer Security*, pages 109–128, Cham, 2019. Springer International Publishing.
- [BKL16] Christof Beierle, Thorsten Kranz, and Gregor Leander. Lightweight Multiplication in $GF(2^n)$ with Applications to MDS Matrices. In Matthew Robshaw and Jonathan Katz, editors, *Advances in Cryptology – CRYPTO 2016*, pages 625–653, Berlin, Heidelberg, 2016. Springer Berlin Heidelberg.
- [BLS⁺17] Jian Bai, Ting Li, Yao Sun, Dingkan Wang, and Dongdai Lin. The Lightest 4 x 4 MDS Matrices over $GL(4, F_2)$. 2017.
- [BMP08] Joan Boyar, Philip Matthews, and René Peralta. On the Shortest Linear Straight-Line Program for Computing Linear Forms. In Edward Ochmański and Jerzy Tyszkiewicz, editors, *Mathematical Foundations of Computer Science 2008*, pages 168–179, Berlin, Heidelberg, 2008. Springer Berlin Heidelberg.
- [BP10] Joan Boyar and René Peralta. A New Combinational Logic Minimization Technique with Applications to Cryptology. pages 178–189, 05 2010.
- [BP18] Joan Boyar and René Peralta. C++ Implementation of SLP Algorithm. 2018.
- [BR03] Paulo Barreto and Vincent Rijmen. The Whirlpool Hashing Function. 24, 01 2003.
- [DL18] Sébastien Duval and Gaëtan Leurent. MDS Matrices with Lightweight Circuits. *IACR Transactions on Symmetric Cryptology*, 2018, 06 2018.
- [DR02] Joan Daemen and Vincent Rijmen. *The Design of Rijndael: AES - The Advanced Encryption Standard*. 01 2002.
- [FSK10] Carsten Fuhs and Peter Schneider-Kamp. Synthesizing Shortest Linear Straight-line Programs over $GF(2)$ using SAT. In *International Conference on Theory and Applications of Satisfiability Testing*, pages 71–84. Springer, 2010.
- [GKM⁺09] Praveen Gauravaram, Lars R. Knudsen, Krystian Matusiewicz, Florian Mendel, Christian Rechberger, Martin Schläffer, and Søren S. Thomsen. Grøstl - a SHA-3 Candidate. In Helena Handschuh, Stefan Lucks, Bart Preneel, and Phillip Rogaway, editors, *Symmetric Cryptography*, volume 9031 of *Dagstuhl Seminar Proceedings (DagSemProc)*, pages 1–33, Dagstuhl, Germany, 2009. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- [JBKK25] Yongjin Jeon, Seungjun Baek, Giyoon Kim, and Jongsung Kim. A Framework for Generating S-Box Circuits with Boyar–Peralta Algorithm-Based Heuristics, and Its Applications to AES, SNOW3G, and Saturnin. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(1):586–631, 2025.

- [JPST17] Jérémy Jean, Thomas Peyrin, Siang Meng Sim, and Jade Tourteaux. Optimizing Implementations of Lightweight Building Blocks. *IACR Transactions on Symmetric Cryptology*, 2017(4):130168, Dec. 2017.
- [JV04] Pascal Junod and Serge Vaudenay. FOX: A New Family of Block Ciphers. volume 3357, 08 2004.
- [KLSW17] Thorsten Kranz, Gregor Leander, Ko Stoffelen, and Friedrich Wiemer. Shorter Linear Straight-Line Programs for MDS Matrices. *IACR Transactions on Symmetric Cryptology*, 2017(4):188211, Dec. 2017.
- [LS16] Meicheng Liu and Siang Meng Sim. Lightweight MDS Generalized Circulant Matrices. In Thomas Peyrin, editor, *Fast Software Encryption*, pages 101–120, Berlin, Heidelberg, 2016. Springer Berlin Heidelberg.
- [LSL⁺19] Shun Li, Siwei Sun, Chaoyun Li, Zihao Wei, and Lei Hu. Constructing Low-latency Involutory MDS Matrices with Lightweight Circuits. *IACR Transactions on Symmetric Cryptology*, 2019(1):84117, Mar. 2019.
- [LW16] Yongqiang Li and Mingsheng Wang. On the Construction of Lightweight Circulant Involutory MDS Matrices. In Thomas Peyrin, editor, *Fast Software Encryption*, pages 121–139, Berlin, Heidelberg, 2016. Springer Berlin Heidelberg.
- [LW17] Chaoyun Li and Qingju Wang. Design of Lightweight Linear Diffusion Layers from Near-MDS Matrices. *IACR Transactions on Symmetric Cryptology*, 2017(1):129155, Mar. 2017.
- [LXZZ21] Da Lin, Zejun Xiang, Xiangyong Zeng, and Shasha Zhang. A Framework to Optimize Implementations of Matrices. In Kenneth G. Paterson, editor, *Topics in Cryptology – CT-RSA 2021*, pages 609–632, Cham, 2021. Springer International Publishing.
- [ME19] Alexander Maximov and Patrik Ekdahl. New Circuit Minimization Techniques for Smaller and Faster AES SBoxes. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 91–125, 08 2019.
- [Paa97] C. Paar. Optimized Arithmetic for Reed-Solomon Encoders. In *Proceedings of IEEE International Symposium on Information Theory*, pages 250–, 1997.
- [SKOP15] Siang Meng Sim, Khoongming Khoo, Frédérique Oggier, and Thomas Peyrin. Lightweight MDS Involution Matrices. In Gregor Leander, editor, *Fast Software Encryption*, pages 471–493, Berlin, Heidelberg, 2015. Springer Berlin Heidelberg.
- [SS17] Sumanta Sarkar and Habeeb Syed. Analysis of Toeplitz MDS Matrices. In Josef Pieprzyk and Suriadi Suriadi, editors, *Information Security and Privacy*, pages 3–18, Cham, 2017. Springer International Publishing.
- [Sto16] Ko Stoffelen. Optimizing S-Box Implementations for Several Criteria Using SAT Solvers. In Thomas Peyrin, editor, *Fast Software Encryption*, pages 140–160, Berlin, Heidelberg, 2016. Springer Berlin Heidelberg.
- [TP19] Quan Quan Tan and Thomas Peyrin. Improved Heuristics for Short Linear Programs. *Cryptology ePrint Archive*, Paper 2019/847, 2019.

- [WSL⁺25] Zihao Wei, Siwei Sun, Fengmei Liu, Lei Hu, and Zhiyu Zhang. Technology-dependent synthesis and optimization of circuits for small s-boxes. *IACR Communications in Cryptology*, 1(4), 2025.
- [XZL⁺20] Zejun Xiang, Xiangyong Zeng, Da Lin, Zhenzhen Bao, and Shasha Zhang. Optimizing Implementations of Linear Layers. Cryptology ePrint Archive, Paper 2020/903, 2020.
- [YWS⁺24] Yufei Yuan, Wenling Wu, Tairong Shi, Lei Zhang, and Yu Zhang. A Framework to Improve the Implementations of Linear Layers. *IACR Transactions on Symmetric Cryptology*, 2024(2):322347, Jun. 2024.